# Programación competitiva

## Volumen I

4 U F W F O ) B M J N o ' F M J Y ) B M J N

1 S P H S B N B D J Ø

7 P M V N F

. B O V B M Q B S B D P O D V S T

7 B M M B E P M

# Índice general

**Prólogos** . . . . . IX

**Comentarios a las ediciones anteriores** . . . . . XVII

**Prefacio** . . . . . XXI

**Perfil de los autores** . . . . . XXXIII

**Abreviaturas** . . . . . XXXVII

**Índice de tablas** . . . . . XXXIX

**Índice de figuras** . . . . . XLIII

**1. Introducción** . . . . . 1

1 S P H S B N B D J ØO D P N Q F U J U J W B

- P T D P O D V S T P T

0 M J N Q J B E B J O U F S O B D J P O B M E F J O G P S N ¨ U J

*International Collegiate Programming Contest*

0 U S P T D P O D V S T P T E F Q S P H S B N B D J ØO

$ P O T F K P T Q B S B T F S D P N Q F U J U J W P

$ P O T F K P F T D S J C F N ¨ T S ¨ Q J E P

$ P O T F K P J E F O U J m D B S ¨ Q J E B N F O U F F M U J

$ P O T F K P B O B M J [ B F M B M H P S J U N P

$ P O T F K P E P N J O B M P T M F O H V B K F T E F Q S P H

$ P O T F K P E P N J O B F M B S U F E F Q S P C B S F M D

$ P O T F K P Q S ¨ D U J D B Z N ¨ T Q S ¨ D U J D B

$ P O T F K P U S B C B K B F O F R V J Q P Q B S B * $ 1 $

1 S J N F S P T Q B T P T M P T Q S P C M F N B T G ¨ D J M F T

" O B U P N ÓB E F V O Q S P C M F N B

3VUJOBTUÓQJDBTEFFOUSBEB TBMJEB
&NQJF[BFMWJBKF
/VFTUSPQSJNFSWFSFEJDUPEFBDFQUBEP "$
)BCJMJEBEFTC¨TJDBTEFQSPDFTBNJFOUPEFDBEF
-PTQaSIPCMFNBT
4PMVDJPOFTBMPTFKFSDJDJPTOPSFTBMUBEPT
/PUBTEFMDBQÓUVMP

**2. Estructuras de datos y bibliotecas** . . . . . **59**
*OUSPEVDDJØO ZNPUJWBDJØO
&TUSVDUVSBTEFEBUPTMJOFBMFTDPOCJCMJPUFD
*Array*
1SPCMFNBTEFPSEFOBDJØO FTQFDJBMFT
.¨TDBSBEFCJUT
*BigInteger*1rZUIPOZ+BWB
&TUSVDUVSBTEFEBUPTFOMB[BEPT
1SPCMFNBTFTQFDJBMFTCBTBEPTFOQJMBT
&TUSVDUVSBTEFEBUPTOPMJOFBMFTDPOCJCMJPU
.POUÓDVMPCJOBSJP DPMBEFQSJPSJEBE
5BackM BEF
«SCPMEFCÞTRVFEBCJOBSJBFRVJMJCSBEP C#
«SCPMEFFTUBEÓTUJDPTEFPSEFO
&TUSVDUVSBTEFEBUPTDPOOVFTUSBTQSPQJBTCJ
(SBGP
$POKVOUPTEJTKVOUPTQBSBVOJØOoCVTDBS
«SCPMEF'FOXJDL CJOBSJPJOEFYBEP
«SCPMEFTFHNFOUPT
4PMVDJPOFTBMPTFKFSDJDJPTOPSFTBMUBEPT
/PUBTEFMDBQÓUVMP

**3. Paradigmas de resolución de problemas** . . . . . **149**
*OUSPEVDDJØO ZNPUJWBDJØO
#ÞTRVFEBDPNQMFUB
#ÞTRVFEBDPNQMFUBJUFSBUJWB
#ÞTRVFEBDPNQMFUBSFDVSTJWB
$POTFKPTTPCSFCÞRVFEBDPNQMFUB
#ÞTRVFEBDPNQMFUBFODPODVSTPTEFQSPHSE
%JWJEFZWFODFS¨T
6TPTEFTUBDBEPTEFMBCÞTRVFEBCJOBSJB

#ÞTRVFEBUFSOBSJB
%JWJEFZWFODFS¨TFODPODVSTPTEFQSPH
7PSB[
&KFNQMPT
"MHPSJUNPTWPSBDFTFODPODVSTPTEFQS
1SPHSBNBDJØOEJO¨NJDB
*MVTUSBDJØOEFQSPHSBNBDJØOEJO¨NJDB
&KFNQMPTDM¨TJDPT
&KFNQMPTOPDM¨TJDPT
1SPHSBNBDJØOEJO¨NJDBFODPODVSTPTEF
4PMVDJPOFTBMPTFKFSDJDJPTOPSFTBMUBEPT
/PUBTEFMDBQÓUVMP

**4. Grafos . . . . . 225**
*OUSPEVDDJØOZNPUJWBDJØO
3FDPSSJEPEFHSBGPT
*OUSPEVDDJØOZNPUJWBDJØO
#ÞTRVFEBFOQSPGVOEJEBE%'4
#ÞTRVFEBFOBODIVSB#'4
#ÞTRVFEBEFDPNQPOFOUFTDPOFYPTHSBG
3FMMFOPQPSEJGVTJØOHSBGPEFSFKJMME
0SEFOUPQPMØHJDPHSBGPBDÓDMJDPEJSJ
$PNQSPCBDJØOEFHSBGPCJQBSUJUPHSBG
$PNQSPCBDJØOEFDJDMPTHSBGPEJSJHJE
#ÞTRVFEBEFQVOUPTEFBSUJDVMBDJØOZQV
#ÞTRVFEBEFDPNQPOFOUFTGVFSUFNFOUF
3FDPSSJEPEFHSBGPTFODPODVSTPTEFQS
«SCPMEFFYQBOTJØONÓOJNP.45
*OUSPEVDDJØOZNPUJWBDJØO
"MHPSJUNPEF,SVTLBM
"MHPSJUNPEF1SJN
0USBTBQMJDBDJPOFT
.45FODPODVSTPTEFQSPHSBNBDJØO
$BNJOPTN¨TDPSUPTEFPSJHFOÞOJDP4441
*OUSPEVDDJØOZNPUJWBDJØO
&OHSBGPTOPQPOEFSBEPT#'4
&OHSBGPTQPOEFSBEPT%JKLTUSB
&OHSBGPTQFRVF×PTDPODJDMPTOFHBUJV
4441FODPODVSTPTEFQSPHSBNBDJØO

$BNJOPTN¨TDPSUPTFOUSFUPEPTMPTQBSFT
*OUSPEVDDJØO ZNPUJWBDJØO
"MHPSJUNPEF ’MPZEo8BSTIBMM
0USBTBQMJDBDJPOFT
"141FODPODVSTPTEFQSPHSBNBDJØO
(SBGPTFTQFDJBMFT
(SBGPBDÓDMJDPEJSJHJEP
«SCPM
(SBGPCJQBSUJUP
(SBGPFVMFSJBOP
(SBGPTFTQFDJBMFTFODPODVSTPTEFQSPHSE
4PMVDJPOFTBMPTFKFSDJDJPTOPSFTBMUBEPT
/PUBTEFMDBQÓUVMP

**Bibliografía** . . . . . **321**

**Índice alfabético** . . . . . **325**

# Prólogos

## Bill Poucher

### Introducción

En 1970, la " . 6 1 & ) P O P S 4 de Texas, EE.UU, albergó el primer concurso universitario de programación de la historia del ICPC. Las primeras finales se celebraron en 1977, en Atlanta, junto a la reunión invernal del congreso de ciencias de la computación de la ACM. El * O U F S O B U J P O B M $ P M M F H J B U F más reciente ha implicado la realización de concursos regionales en 643 sedes de 104 países, en las que han participado 59 000 concursantes junto a sus 5043 preparadores, llegados de más de 3400 universidades de todo el planeta. Los 135 mejores equipos se encontrarán en la final mundial del ICPC, que tendrá lugar en Moscú, con el MIPT como anfitrión, en junio de 2021.

A lo largo y ancho del mundo, hay más de 400 000 estudiantes que han pasado por el ICPC, y muchos de ellos han jugado papeles clave en la construcción de la comunidad digital global durante las últimas décadas. El ICPC es la raiz de la programación competitiva que llega, gracias a esa comunidad global, a personas de todas las culturas y a generaciones cada vez más jóvenes.

El Online Judge de la Universidad de Valladolid abrió las puertas a los concursos en línea y facilitó el acceso global a los problemas del ICPC, bajo la batuta del profesor Miguel Ángel Revilla. Tres de sus usuarios estelares son Steven Halim, Felix Halim y Suhendry Effendy, autores de 1 S P H S B N B D J ± O que, en esta edición, se presenta en dos volúmenes. En la final mundial del ICPC de Moscú, con el MIPT como anfitrión, recibirán un merecido homenaje por parte de la * $ 1 $ ' P V O E B U J P O

### Programación competitiva

¿Qué es la programación competitiva y por qué deberías implicarte en ella? Primero, y más importante, es un deporte mental. Ayuda a desarrollar por completo las capacidades de razonamiento algorítmico y establece un puente fundamental entre la teoría y la práctica. La participación plena desarrolla intuición y competencia a la hora de solucionar problemas. Nos prepara para el 3 F O B D J N J F O U P E F J O G P S N B D J ± O que dará forma al mundo en las próximas décadas. Para apreciar la imagen completa, es importante abrirse paso entre una maraña de terminología insustancial. Es un deporte de equipo.

## ¿Por dónde empezamos?

Empieza con los Volúmenes I y II de 1 S P H S B N B D J ± O. De hecho, sería mejor empezar con el Volumen I :). Los autores son expertos en programación competitiva con una amplia experiencia, y han dedicado décadas de trabajo a ayudar en todas las facetas de este deporte.

En paralelo, implícate en una cultura que desarrolle hábitos de excelencia. Formas parte de la primera generación que nunca ha estado desconectada. Esa conexión alcanza su mejor versión cuando unimos nuestras fuerzas en pos de una causa común. Hazlo y prepárate para afrontar los desafíos que definirán a tu generación.

La vida te necesita. Hemos nacido para competir. Y competimos mejor cuando lo hacemos unidos, con buena fe, con buena voluntad y con buenas acciones. Cuando llegues a la universidad, ten en consideración al ICPC y al nuevo programa * $ 1 $ 6 O J W F S T J U Z , que ofrece un tabanico de actividades para realizar fuera del aula. Encontrarás más detalles en `https://icpc.global`.

## ¿Por qué empezar?

¿Es importante que mejores tus habilidades de resolución de problemas? Sí. ¿Es importante que te prepares para un futuro integrado en una comunidad digital global? Sí. ¿Lo es seguir el consejo de T.S. Eliot de que, para un desarrollo completo, hay que llegar demasiado lejos? Sí. Hazlo en la programación competitiva, pero cuidado con aquellas ambiciones que son irreversibles.

¿Es útil la programación competitiva? Aristóteles afirmaba que no hay nada más útil que dedicarse a actividades mentales y reflexiones que se tienen como fin a sí mismas y marcan su propio ritmo. Permíteme recomendarte que dediques tu alma a construir un mundo más bello. En el inmenso ámbito de la vida, muchos actos de bondad pequeños logran hechos extraordinarios. Encuentra amigos que compartan tus intereses y cíñete a este ciclo:

> Repite toda tu vida: estudio, práctica, ensayo, ensayo general, actuación.

Sirve para los atletas.
Sirve para los músicos.
Sirve para toda clase de intérpretes.
Sirve para tí.

Con mis mejores deseos, Bill.

Dr. William “Bill” Poucher, Ph. D., ACM Fellow
Profesor de ciencias de la computación, Universidad de Baylor
Director ejecutivo, * O U F S O B U J P O B M $ P M M F H J B U F 1 S P H S B N N J O H $ P O U F T U
Presidente, * $ 1 $ ' P V O E B U J P O
13 de julio de 2020

## Miguel Revilla Rodríguez

Hace casi 20 años (el 11 de noviembre de 2003, para ser exactos), mi padre, Miguel Ángel Revilla, recibió un correo electrónico con el siguiente mensaje:

> "Debo decirle, en pocas palabras, que con el 67B 0OMJOF ha nacido una nueva CIVILIZACIÓN y que, con los libros que escribe (se refería a •%FTBG«PT EF QSPHSI NBDJ[53], junto a Steven S. Skiena), inspira a los soldados para que sigan marchando. Le deseo una larga vida de servicio a la Humanidad, para que pueda crear programadores sobrehumanos."

Lo que, en palabras de mi padre, era "una exageración evidente", dio que pensar. Y no es ningún secreto que los pensamientos se convierten en sueños con cierta facilidad. Su sueño era crear una comunidad alrededor del proyecto que había iniciado, como parte de su labor docente en la Universidad de Valladolid, que reuniese a personas de todo el planeta que trabajasen por un mismo ideal. Tras un breve búsqueda, en aquella primitiva internet de los primeros años de nuestro siglo, afloró una extensa comunidad de usuarios y herramientas excepcionales, construidas alrededor de su página web 67B 0OMJOF.

El sitio .FUIPET UP4PMWF[1], creado por un joven estudiante de Indonesia, era uno de los más impresionantes. Allí se podía encontrar el resultado del duro trabajo de un auténtico genio de la algoritmia y la informática. Con ello, quedaba plantada la semilla de la idea de que los sueños se pueden convertir en realidad. Además, no solo las IPKdBT de ese incipiente árbol encajaban perfectamente, sino que la raíz de ambos proyectos era exactamente la misma: servir a la Humanidad. Aquel joven estudiante, autor del correo electrónico y de la página web que hicieron soñar a mi padre, era Steven Halim. Después, descubriría que Steven no estaba solo en su ambición, pues su hermano menor, Felix, compartía su visión, sus intereses y sus capacidades extraordinarias.

Tras 15 años de fructífera colaboración y, lo que es más importante, de amistad con Steven y Felix, mi padre falleció en 2018. Su trabajo, y sus sueños, nos pertenecen ahora a nosotros, la siguiente generación. Este libro es la prueba viva de que el sueño se ha convertido en realidad.

•/PTF NF PDVSSF VO DPNQMFNFOUP, sobre palabras de mi padre. Ahora, con esta cuarta versión de 1SPHSBNBDJ±O en mis manos, me atrevo a añadir que no podría concebir la mera existencia del 0OMJOF sin este libro. Ambos proyectos han crecido en paralelo y son, sin duda, complementos y compañeros perfectos. Al practicar y dominar la mayoría de los ejercicios de programación de este libro, el lector aprenderá a resolver cientos de tareas y encontrar un puesto entre los 500 mejores programadores del 0OMJOF. Tienes en tus manos más de 2000 (sí, ¡dos mil!) problemas del 0OMJOF cuidadosamente elegidos, clasificados y comentados.

Los autores, durante las dos últimas décadas, han crecido desde concursantes a preparadores y, finalmente, maestros en el arte de la programación competitiva. Conocen a la perfección cada cruce y cada recodo de ese largo camino, y son capaces de ponerse en la piel del joven concursante de la IOI, del recién llegado al ICPC o del experto entrenador, hablándole a cada uno en su propio idioma. Por esa razón, este libro es la lectura perfecta para todos ellos. Poco importa si te estás iniciando en la programación competitiva en tu IOI local, o si eres el preparador de un equipo para la próxima final mundial del ICPC, no tengas ninguna duda: ESTE es tu libro.

[1] Se puede visitar en `https://cpbook.net/methodstosolve`.

Me encantan las películas, adoro el cine clásico, y soy consciente de que estoy ante una obra maestra cuando, al terminar, estoy deseando volver a verla. En palabras de Steven y Felix, tendrás esa misma sensación, no solo por la recomendación de los autores, sino porque lo leerás y releerás con atención pues, como ocurre con las grandes películas, descubrirás detalles nuevos y estimulantes cada vez que lo hagas. Siguiendo esa lógica, este libro es una obra maestra.

Tengo, además, el inmenso honor de ser el traductor al castellano de esta obra. La traducción requiere de un meticuloso proceso de transformar las palabras conservando el alma. Tienes que pensar como lo haría el autor, y tienes que entender perfectamente no solo las palabras escritas, sino la intención que hay detrás de ellas. Es un ejercicio de artesanía. Después de recorrer este texto cientos de veces, he disfrutado con cada concepto, con cada idea novedosa, no solo por lo que está escrito en él, sino por lo que pretende lograr. El ideal de formar a mejores programadores y, tras ello, el ideal de servir a la Humanidad. Este libro es, de hecho, una auténtica obra maestra.

Una vez que hayas leído este libro varias veces, descubrirás que eres un programador mucho mejor pero, aunque no lo creas, también descubrirás que eres una persona más feliz.

Miguel Revilla Rodríguez
Administrador del Online Judge
`https://onlinejudge.org`
Valladolid, 1 de julio de 2020

He disfrutado enormemente con la lectura de los sucesivos borradores de esta obra actualizada, que los autores han compartido conmigo en las IOI más recientes, y debo felicitarles por el nivel de excelencia aplicado al ampliar el ámbito, la claridad y la profundidad de un texto que ya era extraordinario.

Brian Christopher Dean
Profesor y presidente
Departamento de ciencias de la computación – Escuela de informática
Clemson University, Clemson, SC, EEUU
Director de la Olimpiada informática de EEUU
5 de julio de 2020
`http://www.usaco.org/`

# Comentarios a las ediciones anteriores

•1SPHSBNBDJ±O DPNQFUJUJWB $1 ‘ IB SFTVMUBEP VOB FOPSNF
FTUSVDUVSBT EF EBUPT Z MPT BMHPSJUNPT 4UFWFO Z 'FMJY
QSPGVOEJEBE UPEPT MPT BTQFDUPT EF MB QSPHSBNBDJ±O D
FOPSNF EF QSPCMFNBT QSŸDUJDPT QBSB BTFHVSBSTF EF RV
NFNPSJB 1SBDUJDBS DPO $1 NF IB BZVEBEP B SFBMJ[BS FOUSF
OVODB QPES§ FTUBS MP TV DJFOUFNFOUF
**— Troy Purvis – Ingeniero de software en Google**

4UFWFO Z 'FMJY TPO VOPT BQBTJPOBEPT EF MB QSPHSBNBDJ±O
VOPT BQBTJPOBEPT FO BZVEBS B MPT FTUVEJBOUFT B DPOWFSU
SFTVMUBEP VOB JONFSTJ±O FO MBT FTUSVDUVSBT EF EBUP
VUJMJ[BEPT QPS QSPHSBNBEPSFT DPNQFUJUJWPT EF UPEP FM
DPHFS QPMWP MB NBZPS FOTF¯BO[B TFSŸ QSPCBCMFNFOU
DPNQMFKB QFSP QVFEFT
**— Dr. Daniel Zingaro – Profesor asociado en la University of Toronto Mississauga**

&M MJCSP •1SPHSBNBDJ±O DPNQFUJUJWB‘ OPT IB BZVEBEP
QBSUJDJQBOUFT FO FM *$1$ Z MB *0* FO #PMJWJB &T MB NFKP
BMDBO[BS VO CVFO OJWFM DPNP QSPHS
**— Jhonatan Castro – preparador de equipos de ICPC e IOI de Bolivia**
**– Universidad Mayor de San Andrés, La Paz, Bolivia**

-B MFDUVSB EF $1 IB TVQVFTUP VOB DPOUSJCVDJ±O GVOEBN
QSPHSBNBEPS DPNQFUJUJWP TJOP UBNCJ§O DPNP DJFOU
SFTPMVDJ±O EF QSPCMFNBT IB NFKPSBEP QSBDUJDBOEP DPO MP
EF SFTPMWFS QSPCMFNBT FTQFDJBMNFOUF FO DPODVSTPT TF
WBSJPT FTUVEJBOUFT VUJMJ[BOEP FTUF MJCSP DPNP HV«B
DVBMRVJFSB RVF RVJFSB NFKPSBS FO TV BDUJV
**— Ryan Austin Fernandez – Profesor ayudante, De La Salle University, Manila, Filipinas**

7PMW« B EFTDVCSJS FM MJCSP $1 FO DVBOEP SFHSFT§
FO #SBTJM %JTGSVU§ BQSFOE« Z SFTPMW« NVDIPT EF TVT QSP
B MB HSBEVBDJ±O BM QSFQBSBS BM UJFNQP RVF TFHV«B
FTUVEJBOUFT OVFWPT DPO JOUFS§T FO MB QSPHSBNBDJ±O DP
DPNQFUFODJB DPO FMMPT Z BM OBM IBO SFTV
**— Luciano Arnaldo Romero Calla – Estudiante de doctorado en la Universidad de Zurich**

$1 NF BZVE± FO MB QSFQBSBDJ±O EVSBOUF MBT GBTFT EF QSFQE
QBSUJDJQBS FO MB *0* $VBOEP DVST§ GPSNBDJ±O TPCSF QSPHSB
/BDJPOBM EF 4JOHBQVS $1 TF VUJMJ[BCB BNQMJBNFOUF QBSB
FYDFMFOUF FRVJMJCSJP FOUSF MPT FKFSDJDJPT EF QSPHSE
DPNQSFOTJ±O IBDFO EF •1SPHSBNBDJ±O DPNQFUJUJWB‘ VO MJCSP
SFGFSFODJB FO DVSTPT BT« DPNP QBSB FM BQSFOEJ[BKF JOEJWJE
QSPHSBNBDJ±O DPNQFUJUJWB MPT FYQFSUPT QVFEFO BQSFOEFS
HSBDJBT B MB WBSJFEBE EF UFNBT QPDP IBCJUVBMFT RVF
**— Jonathan Irvin Gunawan – Ingeniero de software en Google**

&M %S 4UFWFO )BMJN IB TJEP VOP EF MPT NFKPSFT QSPGFTPSFT RV
EF 4JOHBQVS 4VT JOUVJUJWBT WJTVBMJ[BDJPOFT Z MB DMBSJ
BMUBNFOUF DPNQMFKPT OPT GBDJMJUBSPO FOPSNFNFOUF MB DP
OVODB IF MMFHBEP B JNQMJDBSNF UPUBMNFOUF FO MB QSPHS
FOTF¯BO[BT NF SFTVMUBSPO WJUBMFT FO FOUSFWJTUBT EF US
QSPHSBNBEPS 3FDPNJFOEP DPO WFIFNFODJB FTUB OVFWB FEJDJ±
CVFOB JNQSFTJ±O FO VOB FOUSFWJTUB EF USBCBKP
**— Patrick Cho – Científico de aprendizaje de máquinas en Tesla**

'SBDBT§ FTUSFQJUPTBNFOUF FO MB *0* EF OP MMFHBOEP B
QVFTUP &OUPDFT B QSJODJQJPT EF 4UFWFO )BMJN NF SFHBM
RVF NF MMFW± B PCUFOFS VOB NFEE
**— Joey Yu – Estudiante, University of Waterloo, medallista de oro en la IOI de 2018**

$PNP BVUPEJEBDUB OPWBUP •1SPHSBNBDJ±O DPNQFUJUJWB‘ NF BZ
EJWFSUJEB Z EFTB BOUF $PNP QSPHSBNBEPS DPNQFUJUJWP ŸWJEI
FODPOUSBS VOB FOPSNF WBSJFEBE Z EJWFSTJEBE EF QSPCMFNBT
QSFQBSBS DPO BOUFMBDJ±O QBSB NJT FTUVEJBOUFT MPT NBUFS
OFDFTBSJPT QBSB MB DPNQFUJDJ±O )BCJFOEP DPOPDJEP FTPT
QVFEP EFDJS TJO NJFEP B FRVJWPDBSNF RVF •1SPHSBNBDJ±O DPNC
QBSB DPOWFSUJSTF FO VO NBFTUSP
**— Ammar Fathin Sabili – Estudiante de doctorado**
**– Universidad Nacional de Singapur**

-MFWP USFT B¯PT JOWPMVDSBEP FO MB QSPHSBNBDJ±O DPNQFUJUJ
QSJODJQJPT EF "EPSP MBT FTUSVDUVSBT EF EBUPT Z MPT H
TVQPOF VO TBMUP BCJTNBM EF DPOPDJNJFOUP Z MP EJHP FO TF
QSŸDUJDBT FO #*/64 Z UBNCJ§O DPOPD« BM %S 4UFWFO EVSBC
4JOHBQVS &M RVF BNCPT BVUPSFT BVUPHSB BTFO FTUF MJCSP
JNCPSSBCMFT EF NJ WJEB "VORVF BIPSB ZB FTU§ MMFOP EF NBSDBT
GBWPSJUPT .JM HSBDJBT QPS MMFWBSNF B
**— Hocky Yudhiono – Estudiante en la Universidad de Indonesia**

$PNQS§ $1 FM EF BCSJM EF DPNP SFHBMP B N« NJTNP DPO
TJEP MPT E±MBSFT NFKPS HBTUBEPT FO NBUFSJBM FEVDBUJW
DPNQFUJS FO MBT OBMFT NVOEJBMFT EF MB *0* Z EFM *$1$ $SF
FO FODFOEFS MB NFDIB EFM JOUFS§T Z FO QSPQPSDJPOBSNF
UFNBT FTFODJBMFT QBSB MB QSPHSE

**— Sidhant Bansal – Estudiante en la Universidad Nacional de Singapur**

4JFNQSF RVJTF JNQMJDBSNF FO MB QSPHSBNBDJ±O DPNQFUJU
FNQF[BS $POPD« FTUF MJCSP NJFOUSBT DVSTBCB MB BTJHOBUV
MB 6OJWFSTJEBE /BDJPOBM EF 4JOHBQVS DPNP FTUVE
FYUSFNBEBNFOUF ¸UJM QBSB FM BQSFOEJ[BKF EF MB QSPHSB
DPOKVOUP EF FKFSDJDJPT EF ¸BUUJT &TUF MJCSP QSPQPSDJP
EF MB QSPHSBNBDJ±O DPNQFUJUJWB Z QVFEF SFTVMUBS NVZ
FYQFSUPT "M JHVBM RVF $-34 MP FT QBSB MPT BMHPSJUNPT •
QBSB MB QSPHSBNBDJ±O D

**— Jay Ching Lim – Estudiante en la University of Waterloo**

.JT SFDVFSEPT EF $1 FTUŸO BTPDJBEPT B MFFSMP FO DVBMRV
CJCMJPUFDB MB TFEF EF MPT DPODVSTPT /P QBT± NVD
QSPHSBNBDJ±O DPNQFUJUJWB MFZFOEP $1 IBTUB RVF NF DM

**— Javier Eduardo Ojeda Jorge – Finalista mundial del ICPC**
**– Universidad Mayor de San Simón**
**– Ingeniero de software en dParadig, Chile**

# Prefacio

&TUF EFCF TFS FM MJCSP EF DBCFDFSB EF UPEP QSPHSBNBEPS I
MB QSFTFOUF PCSB FT VOB DPOEJDJ±O OFDFTBSJB BVORVF RVJ
TFS VO QSPHSBNBEPS OPSNBM B DPOWFSUJSTF FO VOP EF MPT N

&M 7PMVNFO * EF FTUF MJCSP TF EJSJHF B MB TJHVJFOUF BVEJFO

&TUVEJBOUFT EF TFDVOEBSJB RVF DPNQJUFO FO MB 0MJNQ
*0* <> JODMVZFOEP TVT GBTFT MPDBMFT QVFT FTUF 7PMV
BDUVBM UFNBSJP>EF MB *0* <

&TUVEJBOUFT VOJWFSTJUBSJPT RVF EFTFFO VUJMJ[BS FTUF
DVSTPT U«QJDPT EF FTUSVDUVSBT EF EBUPT Z BMHPSJUNPT

$VBMRVJFSB JOUFSFTBEP FO QSFQBSBSTF QBSB MB TFDDJ±
BMHPSJUNPT FO MBT FOUSFWJTUBT EF USBCBKP FO FNQSFTB

-PT MFDUPSFT B RVJFOFT WPMÚ TUFOEFM MJCSP JODMVZFO B

&TUVEJBOUFT VOJWFSTJUBSJPT RVF DPNQJUFO FO MB *Intercontinental Collegiate* GBTF
*Programming Contest* < > Z FO MB QSPQJB SOBM NVOEJBM QVFT FM 7PM
EF UFNBT SFMBDJPOBEPT DPO MBT DJFODJBT EF MB DPNQVUB

1SFQBSBEPSFT RVF CVTDBO NBUFSJBM EF GP>SNBDJ±O QBSB

$VBMRVJFSB JOUFSFTBEP FO MB SFTPMVDJ±O EF QSPCMFNB
DPODVSTPT B MPT RVF QVFEFO BDDFEFS RVJFOFT ZB RVFEB
USF *Google CodeJam Facebook Hacker Cup TopCoder Open* DPODVSTPT *ACM ICPC World Finals* EF
*Internet Problem Solving Contest* 14$ FUD

## Requisitos previos

&TUF MJCSP TF EJSJHF B QSPHSBNBEPSFT OPWBUPT MP RV
BTQFDUPT DPODSFUPT EF MB QSPHSBNBDJ±O DPNQFUJUJWB FO
HSBNBDJ±O RVF ZB FTUÝO JODMVJEPT FO PUSPT MJCSPT EF UFY
PCKFUJWP TPO MFDUPSFT RVF UJFOFO BM NFOPT DPOPDJNJFO
NBDJ±O FTUÝO GBNJMJBSJ[BEPT DPO DPNP N«OJNP VOP EF MP

+BWB 1ZUIPO Z 0$BNM BVORVF FT QSFGFSJCMF RVF DPOP[DBO WB
DJ±O CŸTJDB FO FTUSVDUVSBT EF EBUPT Z BMHPSJUNPT FTUBT NB
FOUSF MBT RVF TF JNQBSUFO FO FM QSJNFS B¯P EF JOHFOJFS«B JC
FO MPT DBNQVT EF FOUSFOBNJFOUP QBSB MB *0* Z DPNQSFOEFO E
NFOPT MB OP-0 BDJ±O MBT TJHVJFOUFT TFDDJPOFT JEFOUJSDBSFNPT
EF FTUF UFYUP

## A los (futuros) concursantes de la IOI

-B *0* OP FT VO DPODVST*Español* NVDIPT EF FMMPT USBUBEPT FO FM 7PMVNFO ** 1VFEFT JHOPSBS
MB VOJWFSTJEBE DVBOEP UF VOBT B MPT FRVJQPT VOJWFSTJUBSJ
FTUBT U§DOJDBT QPS BEFMBOUBEP QPES«B SFTVMUBS CFOFSDJPTF
TFSŸO NŸT TFODJMMBT DPO DPOPDJNJFOUPT BEJDJPOBMFT 1PS FM
QSPHSBNBDJ±O DPNQFUJUJWB UF SFDPNFOEBNPT RVF UF IBHBT DP
QPTJCMF FT EFDJS NJFOUSBT FTU§T FO TFDVOEBSJB

4BCFNPT RVF OP TF QVFEF HBOBS VOB NFEBMMB EF MB *Versión* *actual* EF FTUF MJCSP "VORVF DPOSBNPT FO IBCFS JODMVJEP FTQFD
NFO NVDIBT EF MBT NBUFSJBT EF VOB QVOUVBDJ±O SFTQFUBCMF TPNPT NVZ DPOTDJFOUFT EF RVF M
DBQBDJEBEFT EF SFTPMVDJ±O EF QSPCMFNBT FYDFQDJPOBMFT BT«
WJSUVEFT RVF TPO JNQPTJCMFT EF USBOTNJUJS B USBW§T EF VO MJ
DJPOBS DPOPDJNJFOUPT QFSP FM USBCBKP EVSP UF DPSSFTQPOE
MB FYQFSJFODJB Z DPO MB FYQFSJFODJB TF BERVJFSF MB DBQBDJE

| Temas incluidos en el Volumen II |
|---|
| Matemáticas: *Big Integer*, inverso modular, teoría de probabilidad, teoría de juegos |
| Procesamiento de cadenas: árboles y *arrays* de sufijos, KMP, *hashing* de cadenas/Rabin–Karp |
| Geometría (computacional): varias rutinas de biblioteca específicas |
| Grafos: flujo de red, problemas de emparejamiento más difíciles, problemas NP-complejo/completo poco habituales |
| Más de la mitad de los temas pocos habituales |

5BCMB BUFS*Idioma* OP JODMVJEBT FO FM UFYUP EF MB *0* <

## A alumnos de cursos sobre *estructuras de datos y algoritmos*

&M DPOUFOJEP EF FTUF MJCSP IB TJEP FYUFOEJEP FO *cuatro primeros* DBQ«UVMPT SFTVMUFO NŸT BDDFTJCMFT QBSB *primer año* FKFSDJDJPT RVF IFNPT DPOTJEFSBEP EF NBZPS EJSDVMUBE Z QPS
UBEPSFT QBSB MPT QSJODJQJBOUFT TF IBO USBTMBEBEP BM 7PMV
RVF DPNJFO[BO TVT FTUVEJPT EF JOGPSNŸUJDB OP TF TFOUJSŸO J
QSJNFS WPMVNFO

&M DBQ«UVMP JODMVZF VOB DPMFDDJ±O EF QSPCMFNBT EF QSPHS
SFTPMWFS FTUVEJBOUFT EF DJFODJBT EF MB DPNQVUBDJ±O RVF FT
DVSTP CŸTJDP EF NFUPEPMPH«B EF QSPHSBNBDJ±O

&M DBQ«UVMP IB TJEP BDUVBMJ[BEP QSPGVOEBNFOUF "IPSB I
FTUSVDUVSBT EF EBUPT MJOFBMFT Z OP MJOFBMFT DPO NVDIPT
QVFEB VUJMJ[BS UBNCJ§O DP*NPCBTFEFEVTDVSTPUEEP FO MPT E
SFMBUJWPT BMB

-PT DVBUSP QBSBEJHNBT EF SFTPMVDJ±O EF QSPCMFNBT USBUBE
UFNFOUF FO DVGSTPT EM UFYUP EF FTUF DBQ«UVMP IB TJEP BNQM
BZVEBS B MPT FTUVEJBOUFT SFDJ§O MMFHBEPT B MB JOGPSNŸU

)BZ TFDDJPOFT EFM DBQ«UVMP RVF UBNCJ§O QVFEFO VUJMJ[B
QBSB NFKPSBS MB < > P QBSB VO DVSTP CŸTJDP
EJP 5BNCJ§O IFNPT JODMVJEP QVOUPT EF WJTUB SFMBUJ
U§DOJDBT EF QSPHSBNBDJ±O EJOŸNJDB DPNP BMHPSJUNPT FO
TJFOEP NBUFSJBT QPDP DPNVOFT FO NVDIPT UFYUPT TPCSF DJF

## A quienes se preparan para una entrevista de trabajo

&T BNQMJBNFOUF DPOPDJEP RVF NVDIBT FOUSFWJTUBT EF USBC
M±HJDBT JODMVZFO QSFHVOUBT GVOEBNFOUBMFT TPCSF FTUSV
EF JNQMFNFOUBDJ±O &O FTUF QSJNFS WPMVNFO TF USBUBO NV
NVDIB TVFSUF FO UV FOUSFWJTUB

&O FM PUSP MBEP EF MB FOUSFWJTUB NVDIPT FOUSFWJTUBEPSF
QSFHVOUBS RVF SFBMJ[BSŸO

## A los concursantes del ICPC

4PJT MPT MFDUPSFT NŸT FWJEFOUFT TF FOGSFOUBSŸO DF

4BCFNPT RVF QSPCBCMFNFOUF TFSŸ EJG«DJM HBOBS VOB GBT
DPOUFOJEP EF FTUF MJCSP "VORVF IFNPT JODMVJEP NVDIPT NE
FEJDJPOFT BOUFSJPSFT TPNPT DPOTDJFOUFT EF RVF TF SFRVJ
QBSB MPHSBS UBM PCKFUJWP -BT OPUBT EF MPT DBQ«UVMPT DP
MMPT MFDUPSFT IBNCSJFOUPT EF NŸT DPOPDJNJFOUP $SFFNPT
SFTVMUBEPT NVDIP NFKPSFT FO GVUVSPT DPODVSTPT EFTQV§T
RVF FTUF MJCSP TJSWB UBOUP EF JOTQJSBDJ±O DPNP EF NPUJW
DPODVSTBOUF FO FM *$1$ EVSBOUF UV FUBQB VOJWFSTJUBSJB

## A los profesores y preparadores

&TUF MJCSP TF VUJMJ[B FO FM DVSTP $4 ‡Ž1SPHSBNBDJ±O DP
MB &TDVFMB EF *OGPSNŸUJDB EF MB 6OJWFSTJEBE /BDJPOBM E
MP MBSHP EF TFNBOBT MFDUJWBT VUJMJ[BOEP TPMP MBUFSJBM
QSFQBSBEPSFT RVF VUJMJDFO FM DVSTP DPNP SFGFSFODJB U
FTUVEJPT QBSB BKVTUBSMP B MBT OFDFTJEBEFT EF TVT BMVNO
FODPOUSBS QJTUBT P TPMVDJPOFT CIFWFT EFMPTFKFSDJDJPT
DFO FO FM MJCSP "MHVOPT EFMPTFKFSDJDJPT TPO NVZ FTG«DJ

| Semana | Tema | En este libro |
|---|---|---|
| 1 | Introducción | Capítulo 1 |
| 2 | Estructuras de datos y bibliotecas | Capítulos 2 y 9 |
| 3 | Búsqueda completa | Capítulos 3, 8 y 9 |
| 4 | Programación dinámica | Capítulos 3, 8 y 9 |
| 5 | Periodo comodín | Capítulos 3, 4, 9 y otros |
| 6 | Concurso por equipos a mitad del semestre | Todo el volumen I |
| - | Vacaciones a mitad del semestre | - |
| 7 | Grafos 1 (flujo de red) | Capítulos 8 y 9 |
| 8 | Grafos 2 (emparejamiento) | Capítulos 8 y 9 |
| 9 | Problemas NP-complejo/completo | Capítulo 8 |
| 10 | Matemáticas | Capítulos 5 y 9 |
| 11 | Procesamiento de cadenas (*array* de sufijos) | Capítulo 6 |
| 12 | Geometría (computacional) (bibliotecas) | Capítulos 7 y 9 |
| 13 | Concurso por equipos final | Ambos volúmenes y más |
| - | No hay examen final | - |

5BCMBMBO EF FTUVEJPT EFM DVSTP $4 EF 4UFWFO OJWFM QBSB

Z OP UJFOFO OJ QJTUBT OJ TPMVDJPOFT 4F QPES«BO VUJMJ[BS DPN
QBSB VO DPODVSTP PCWJBNFOUF FM QSPGFTPS EFCFSŸ SFTPMWFS

## A todos los lectores

%BEB TV EJWFSTJEBE FO DPCFSUVSB Z QSPGVOEJEBE EF «UESPBVOBN JF
¸OJDB WF[ TJOP RVF QSFUFOEF TFS DPOTVMUBEP DO GSFDVFODJE
DJPT FTBSJUPZTEF QSPHSBNBDJ±OJTUSJCVJEPT FOUSF DBTJ UPEBT
1VFEFT JHOPSBS EJDIPT FKFSDJDJPT FO VOB QSJNFSB BQSPYJNBDJ
SFRVJFSFO NŸT U§DOJDBT Z BQSFOEJ[BKF Z WPMWFS TPCSF FMMP
MFDUVSB 3FTPMWFS FTPT FKFSDJDJPT GPSUBMFDFSŸ UV DPNQSFO
OPSNBMNFOUF JNQMJDBO BQMJDBDJPOFT JOUFSFTBOUFT Z WBSJE
UFOUBS SFTPMWFSMPT FO BMH¸O NPNFOUP ZB RVF IBDFSMP OP TFS

$SFFNPT RVF FTUF MJCSP FT Z TFSŸ SFMFWBOUF QBSB NVDIPT FTUV
SJPT JODMVTP QBSB RVJFOFT ZB IBO SOBMJ[BEP TVT FTUVEJPT QFS
BSDJ±O -PT DPODVSTPT EF QSPHSBNBDJ±O DPNP FM *$1$ P MB *0*
NFOPT EVSBOUF NVDIPT B¯PT -PT OVFWPT FTUVEJBOUFT EFCFO JOU
DPOPDJNJFOUPT CŸTJDPT QSFTFOUBEPT BOUFT EF CVTDBS OVFWPT
DPTŒ OP EFCF MMFWBS B DPOGVTJ±O DPNP QPESŸT DPNQSPCBS EF

&M QSPQ±TJUP EF FTUF MJCSP FT DMBSP RVFSFNPT NFKPSBS MBT D
MVDJ±O EF QSPCMFNBT EF UPEPT*ubVreTtlisFtoFUMSFTDZPDPVOSTFMMEF QSP
NBDJ±O DPNP FM *$1$ P MB *0* $PO NŸT DPODVSTBOUFT EPNJOBO
FTQFSBNPT RVF FM B¯P DVBOEP TF QVCMJD± MB QSJNFSB FEJD
YJ±O RVF SFTVMUF FO VOB NFKPSB BDFMFSBEB EF MPT FTUŸOEBSF
&TQFSBNPT BZVEBS B DPODVSTBOUFT B ≥MPH±SBM NFKPS FT QVOUVBD
QVOUPT SFTPMWJFOEP UPEBT MBT TVCUBSFBT EF MBT UBSFBT E
*0* Z B NŸT FRVJQPT B≥SF±TBMWFS NŸT QSPCMFNBT *problemas regulares* FM

RVF TVFMF BQBSFDFS FO DBEB DPODVSTP QSPCMFNBT FO MPT G
NVDIPT QSFQBSBEPSFT EFM *$1$ Z EF MB *0* EF UPEP FM NVOEP
RVF QSPQPSDJPOB FO FM EPNJOJP EF UFNBT RVF MPT FTUVEJBOU
DPODVSTPT EF QSPHSBNBDJ±O 4J MPHSBNPT RVF OVFTUSB QSF
‹N«OJNPŒ SFRVFSJEP FO MB QSPHSBNBDJ±O DPNQFUJUJWB DPC
[BEP OVFTUSP PCKFUJWP EF FMFWBS FM OJWFM EFM DPOPDJNJF
FTUF MJCSP SFDJCJSFNPT MB NBZPS TBUJTGBDDJ±O QPS FMMP

## Convenciones

&O FTUF MJCSP TF JODMVZF NVDIP D±EJHP GVFOUF FO $ $ +B
0$BNM $VBOEP BQBSF[DB FT UBGrafía nombespaciado STUB FM D±EJH
FTUŸ JOEFOUBEP FO FTDBMPOFT EF FTQBDJPT TBMWP FM FTDS

&O FM D±EJHP GVFOUF EF $ $ IFNPT BEPQUBEF PZ FMB VS TG SDFB V
SBDUFS«TUJDBT RVF TF VUJMJ[BO IBCJUVBMN FOUF QPG QSPHSE
*brevedad y velocidad en la programación*4JO FNCBSHP OP TJFNQSF FT QPTJCMF
OJDBT FO +BWB 1ZUIPO V 0$BNM ZB RVF OP UJFOFO DBSBDUFS
FKFNQMPT EF OVFTUSPT BUBKPT QBSB $ $

```
typedef long long ll;
typedef pair<int, int> ii;
typedef vector<int> vi;
typedef vector<ii> vii;
memset(memo, -1, sizeof memo);
vi memo(n, -1);
memset(arr, 0, sizeof arr);
```

*typedef* *long* *long* ll; MPT DPNFOUBSJPT EF UJQPT EF EBUPT DPNV
*typedef* pair<*int*, *int*> ii; RVF BQBSFDFO NF[DMBEPT DPO FM D³EJ
*typedef* vector<*int*> vi; BQBSFDFO KVTUJGJDBEPT B MB EFSFDI
*typedef* vector<ii> vii;
memset(memo, -1, *sizeof* memo); JOJDJBMJ[BS UBCMB EF NFNPJ[BDJ³O EF %
vi memo(n, -1); N©UPEP BMUFSOBUJWP
memset(arr, 0, *sizeof* arr); WBDJBEP EF VO BSSBZ EF FOUFSPT

-PT TJHVJFOUFT BUBKPT BQBSFDFO GSFDVFOUFNFOUF FO OVFTU
TPO BQMJDBCMFT B 1ZUIPO V 0$BNM

```
   "UBKPT QBSB DPOTUBOUFT DPNVOFT
const int INF = 1e9;
const int LLINF = 4e18;
const double EPS = 1e-9;
++i;
ans = a ? b : c;
ans += val;
index = (index+1) % n;
index = (index+n-1) % n;
int ans = (int)((double)d + 0.5);
ans = min(ans, new_computation);
   IBZ D³EJHP RVF VUJMJ[B    "/%  Z ]] 03
   IBZ D³EJHP RVF VUJMJ[B W-ODVMPT FTUSVDUVSBEPT EF $    QBSB Q
   OP VUJMJ[BNPT MMBWFT FO CVDMFT P DPOEJDJPOBMFT EF VOB M-OF
   QBTBNPT MPT BSHVNFOUPT QPS SFGFSFODJB    TJFNQSF RVF FT QPT
```

"¯BEJNPT NŸT DBSBDUFS«TUJDBT EF MB FTUSVDUVSB EF EBU
EF 'FOXJDL DPNP VOB WBSJBOUF EF FTUSVDUVSB EF EBUPT
DPNP WBSJBOUFT EF MBT DPOTVMUBT EF BDUVBMJ[BDJ±O EF
EF SBOHP QPS SBOHP

"¯BEJNPT NŸT DBSBDUFS«TUJDBT QBSB MB FTUSVDUVSB EF E
UVBMJ[BDJ±O EF SBOHPT DPO QSPQBHBDJ±O QFSF[PTB QBSI
0(MRH)H

- $BNCJPT FO FM DBQ«UVMP

  )FNPT B¯BEJEP EPT U§DOJDBT NŸT EF C¸TRVFEB DPNQMFUB D
  SFTQVFTUBT Z WFSJSDBDJ±O EF UPEBT MBT SFTQVFTUBT QP
  NFEJBOUF C¸TRVFEB CJOBSJB P DVBOEP FM QPTJCMF SBOHI
  5BNCJ§O BDUVBMJ[BNPT MB JNQMFNFOUBDJ±O EF MB NŸTDBS
  MJDF MB LUSHe OTJJDFBNQSF RVF TFB QPTJCMF )FNPT JODMVJEP N
  C¸TRVFEB DPNQMFUB DPNP DPNQSFTJ±O EF EBUPT QBSB IBDF
  VUJMJ[BSMB 5BNCJ§O IFNPT QSPCBEP 1ZUIPO FO QSPCMFNBT
  RVF FO MB NBZPS«B EF MPT QSPCMFNBT EF C¸TRVFEB DPNQMF
  TFSŸ 5-& IBZ GPSNBT EF VUJMJ[BSMP FO DBTPT NŸT TFODJMM

  /PT IFNPT JODMJOBEP QPS FMVMB EN CMF TUBDJ±O EF MB C
  CJOBSJB EF MB SFTQVFTUB #4wi'le F5B NEC JSSJNOUHSEFNPT FO
  DBQ«UVMP MB C¸TRVFEB UFSOBSJB

  "IPSB DPOTJEFSBNPT FM QSPCMFNB EF FNQBSFKBNJFOUP CJ
  CMFNB WPSB[DMŸTJDP "¯BEJNPT RVF BMHVOPT BMHPSJUNPT
  EBUPT EF DPMBT EF QSJPSJEJEBE QBSB PSEFOBS EJOŸNJDBN

  6UJMJ[BNPT MB TPMVDJ±O -*4 ‹PSEFOBDJ±O MBDFNFOUFC FO
  TPMVDJ±O QSFEFUFSNJOBEB QBSB QSPCMFNBTLsm4 NPEFSOPT
  EFOUSP EF MB TPMVDJ±O×%1 5MP1 RVF QFSNJUF[SFSTPMWFS
  NŸT SŸQJEP

- $BNCJPT FO FM DBQ«UVMP

  )FNPT SFEJCVKBEP DBTJ UPEBT MBT SHVSBT EFM DBQ«UVMP

  "IPSB VUJMJ[BNPT FM BMHPSJUNP EF ¸PTBSBKV DPNP QSFEFU
  OFOUFT GVFSUFNFOUF DPOFYPT 4$$ ZB RVF SFTVMUB NŸT
  5BSKBO

  )FNPT FYUFOEJEP EF GPSNB TJHOJSDBUJWB MB TFDDJ±O TPCS
  DIBT EF TVT WBSJBOUFT DPOPDJEBT 5SBUBNPT Z DPNQBSBNP
  NFOUBDJ±O EFM BMHPSJUNP EF %JKLTUSB )FNPT USBTMBEB
  OPNCSŸOEPMP exDamsNoEFOBMHPSJUNP EF #FMMNBO‡'PSE Z EF
  BMHPSJUNP EF #FMMNBO‡'PSE‡.PPSF

  )FNPT BNQMJBEP EF GPSNB TJHOJSDBUJWB MB TFDDJ±O TPCS
  FM BMHPSJUNP EF 'MFVSZ DPO FM NŸT FSDJFOUF EF )JFSIPM[I

  "¯BEJNPT OPUBT TPCSF BMHVOPT HSBGPT FTQFDJBMFT NŸT Z

- $BNCJPT FO FM DBQ«UVMP

  "NQMJBNPT FM USBUBNJFOUP TPCSF adMbT QESFOCUMFSNDBTNRBUVFON
  OVFWP QSPCMFNBadNbUSFNDŸVSSJDFOUF MB GSBDDJ±O

3FDPOPDFNPT FM DBNCJP EF UFOEFODJB QPS FM RVF FM
FOUFSPT HSBOEFT FTUŸ TJFOEP TVTUJUVJEP QPS QSPCM
UBOUP IFNPT BNQMJBEP TJHOJSDBUJWBNFOUF FM USBU
MB TFDDJ±O EF UFPS«B EF O¸NFSPT Z MB QSFTFOUBNPT E
TFDDJPOFT DPNP QPS FKFNQMP FM QFRVF¯P UFPSFNB E
NPEVMBS RVF TF VUJMJ[B QBSB MB JNQMFNFOUBDJ±O EF
O¸NFSPT EF $BUBMBO FO MB TFDDJ±O EF DPNCJOBUPSJB
FT MB QSFEFUFSNJOBEB FO MB TFDDJ±O EF QPUFODJB EF

"NQMJBNPT MB TFDDJ±O EF DPNCJOBUPSJB DPO NŸT NFOD

"NQMJBNPT FM USBUBNJFOUP EF QSPCMFNBT SFMBUJWPT

.FKPSBNPT MB FYQMJDBDJ±O EFM BMHPSJUNP EF C¸TRVF
MJFCSF NFEJBOUF MB IFSSBNJFOUB 7JTV"MHP

*OUFHSBNPT FO FTUF DBQ«UVMP MB QPUFODJB EF NBUSJ
JODPSQPSBNPT U§DOJDBT EF BSJUN§UJDB NPEVMBS FO T

- $BNCJPT FO FM DBQ«UVMP

  5SBUBNPT MB %1 EF E«HJUPT DPNP VO OVFWP QSPCMFNB E

  'PSUBMFDFNPT OVFTUSB[ SHFFOQFMSJBMB DJ±OEFMBMHPSJUNP EF TVSKPT

  "¯BEJNPTTFMEFHDBEFOBT DPNP N§UPEP BMUFSOBUJWP EF
  EF QSPDFTBNJFOUP EF DBEFOBT JODMVZFOEP FM USBUB
  DJB EF DBEFOBT EFTEF FTUF DPOUFYUP

  *OUFHSBNPT Z BNQMJBNPT MB TFDDJ±O TPCSF BOBHSBNBT
  EF QSPDFTBNJFOUP EF DBEFOBT DPO EJGFSFOUFT WBSJB

- $BNCJPT FO FM DBQ«UVMP

  .FKPSBNPT MBT SVUJOBT EF CJCMJPUFDB N,

# Índice de tablas


1. Materias *todavía* no incluidas en el temario de la IOI [16] xxii
2. Plan de estudios del curso CS3233 de Steven (nivel para fases regionales del ICPC) xxiv
3. Victorias en ICPC regionales de la UNS en la década de 2010 xxxiv
4. 3 mejores puestos de la UNS en las finales mundiales del ICPC en la década de 2010 xxxiv

1.1. Tipos de problemas en los ICPC regionales de Asia recientes 10
1.2. Tipos de problemas (versión compacta) 10
1.3. Ejercicio: leer y clasificar estos problemas de UVa/Kattis 12
1.4. Referencia rápida del 'peor algoritmo AC' para varios tamaños de entrada $n$ 15

2.1. Ejemplo de emparejamiento de delimitadores (paréntesis) 81
2.2. Ejemplos de expresiones infijas, prefijas y posfijas 81
2.3. Ejemplo de cálculo posfijo 82
2.4. Ejemplo de ejecución del algoritmo del patio de maniobras 83
2.5. Ejemplo de tabla de frecuencia acumulada 121
2.6. Comparativa entre el árbol de Fenwick y el árbol de segmentos 142

3.1. Ejecución del método de bisección en la función de ejemplo 174
3.2. Tabla de decisión de programación dinámica 199
3.3. UVa 00108 - Maximum Sum 203
3.4. Resumen de los problemas de DP clásicos de esta sección 218
3.5. Comparativa de técnicas de resolución de problemas (orientativa) 223

4.1. Lista de terminología de grafos importante 226
4.2. Tabla de decisión de algoritmos de recorrido de grafos 249
4.3. Variantes clásicas de SSSP y problemas de ejemplo 279
4.4. Tabla de decisión del algoritmo para caminos más cortos 291

# Índice de figuras


1.1. Ilustración de UVa 10911 - Forming Quiz Teams . . . . . 2
1.2. Online Judge y Kattis . . . . . 26
1.3. USACO e ICPC Live Archive . . . . . 26
1.4. Algunas de las fuentes que han inspirado a los autores de este libro . . . . . 57

2.1. Visualización de ordenación, ejemplo de la primera partición de la ordenación *quick* . . . . . 64
2.2. Ejemplo de la operación de mezcla en la ordenación por mezcla . . . . . 67
2.3. Visualización de máscaras de bits, ejemplo de la operación `CheckBit(j)` . . . . . 71
2.4. Ejemplo de un montículo binario (máximo) con su elemento máximo (90) destacado . . . . . 88
2.5. Búsqueda de clave 7 en tabla de *hash* con $m = 11$ y usando encadenamiento separado . . . 92
2.6. Ejemplo de búsqueda de una clave (7) en un BST equilibrado (bBST) . . . . . 96
2.7. Visualización de la estructura de datos de un grafo no dirigido y no ponderado . . . . . 109
2.8. Ejemplos de grafos implícitos . . . . . 112
2.9. Estado inicial: 5 conjuntos disjuntos = 5 árboles aislados/vértices individuales . . . . . 116
2.10. Arriba: `findSet(0)` – Abajo: la compresión de caminos resultante . . . . . 116
2.11. `unionSet(0, 1)` ¨ `(2, 3)` ¨ `(4, 3)` e `isSameSet(0, 4)` . . . . 117
2.12. Ejemplo de `rsq(6)` en un árbol de Fenwick (interrogación/consulta) . . . . . 123
2.13. Ejemplo de `rsq(3)` en un árbol de Fenwick (interrogación/consulta) . . . . . 123
2.14. Ejemplo de `update(5, 2)` en un árbol de Fenwick (actualización) . . . . . 124
2.15. Ejemplo de RUPQ: las consultas de punto $i = 14$ están subrayadas . . . . . 126
2.16. Ejemplo de RUPQ: $X$ indica -1, las consultas de punto $i = 14$ están subrayadas . . . . . 127
2.17. Ejemplo de RURQ: las consultas de rango están subrayadas . . . . . 127
2.18. Explicación de RURQ: las consultas de rango están subrayadas . . . . . 128
2.19. Árbol de segmentos de A = {18, 17, 13, 19, 15, 11, 20, 4} y `RMQ(1, 3) = 13` . . . . 134
2.20. Árbol de segmentos de A = {18, 17, 13, 19, 15, 11, 20, 4} y `RMQ(4, 7) = 11` . . . . 135
2.21. Actualización de A a {18, 17, 13, 19, 15, , 20, 4} . . . . . 136
2.22. Actualización de A a { , 15, 77, 20, 4} . . . . . 137
2.23. Actualización de A a {30, 30, 30, , 15, 77, 20, 4} . . . . . 138

3.1. 8-reinas . . . . . 158
3.2. Árbol de recursión del TSP con $n = 5$ . Ver también la figura 4.42 . . . . . 161
3.3. UVa 10360 [45] . . . . . 163
3.4. My Ancestor (los 5 caminos de la raíz a una hoja están ordenados) . . . . . 173
3.5. Función monótona, I: Bisección – D: BSTA . . . . . 175
3.6. Búsqueda ternaria en una función unimodal . . . . . 177

4.39. Problema de los puentes de Königsberg y su representación en un grafo. . . . . 308.
4.40. Ejemplo del algoritmo de Hierholzer . . . . . 310. . . .
4.41. Algunos grafos especiales más . . . . . 311 . . . .
4.42. DAG de recursión del TSP con $n = 5$ , ver también la figura 3.2 . . . . . 318 .
4.43. LIS como camino más largo de un DAG implícito, ver también la figura 3.13 . . . . . 318

# Capítulo 1

# Introducción

*¡Quiero competir en la final mundial del ICPC!*
**— Un estudiante aplicado**

## 1.1 Programación competitiva

La premisa fundamental de la programación competitiva dice así: "enfrentarse a problemas de programación bien conocidos y resolverlos lo más rápidamente posible".

Analicemos los términos. Mencionar 'problemas de programación bien conocidos' implica que, en la programación competitiva, nos enfrentamos a problemas *ya resueltos* y *no* a problemas de investigación (en los que la solución todavía no se conoce). Hay quien ya los ha resuelto antes (como mínimo, el autor del problema). 'Resolverlos' significa que debemos[1] mejorar nuestros conocimientos informáticos hasta el nivel requerido, para ser capaces de producir un código que funcione y que encuentre la solución, al menos, en términos de obtener la *misma* salida en los casos de prueba secretos[2] del autor del problema, dentro del tiempo límite estipulado. La necesidad de resolver los problemas 'lo más rápidamente posible' es, precisamente, donde reside el elemento competitivo, ya que la velocidad es un objetivo muy natural en los humanos.

Hay que tener en cuenta que destacar en la programación competitiva es un medio, y *no* un fn en sí mismo. El auténtico objetivo es formar mejores programadores, que estén más preparados para producir mejor software y para enfrentarse, en un futuro, a retos de investigación más complejos. Este es uno de los objetivos de la Olimpiada internacional de informática (IOI) [3] y la visión de los fundadores del International Collegiate Programming Contest (ICPC) [4]. Con este libro, queremos hacer nuestra pequeña contribución a que las generaciones presentes y futuras sean más competitivas en la solución de los problemas de programación ya conocidos y que solemos encontrar en las ediciones más recientes tanto del ICPC como de la IOI.

[1] En algunos concursos de programación se compite en equipo, alentando así el trabajo en grupo ya que, en la vida real, los ingenieros de software no suelen trabajar solos.

[2] Al ocultar los casos de prueba fnales en el planteamiento del problema, la programación competitiva alienta a quienes tratan de resolverlo a ejercitar su fortaleza mental, para tratar de encontrar todos los casos límite que sean capaces, y comprobar que sus programas los resuelven. Esta es una situación típica para los ingenieros de software, que deben probar sus programas en profundidad, hasta estar seguros de que cumplen con los requisitos establecidos por los clientes.

## Ejemplo

### Problema número 10911 del UVa Online Judge [45] (Forming Quiz Teams)

**Enunciado resumido del problema**

Tenemos $(x,y)$ como coordenadas de la casa de un estudiante en un plano bidimensional. Hay $2N$ estudiantes y queremos **emparejarlos** en $N$ grupos. $d_i$ será la distancia entre las casas de 2 estudiantes en el grupo $i$. Formar $N$ grupos de forma que $cost = \sum_{i=1}^{N} d_i$ sea **mínimo**. Devolver el valor $cost$ mínimo. Restricciones: $1 \le N \le 8$ y $0 \le x,y \le 1000$.

**Ejemplo de entrada (con explicación)**

$N = 2$; las coordenadas de las $2N = 4$ casas son {1, 1}, {8, 6}, {6, 8} y {1, 3}.

**Ejemplo de salida (con explicación)**

$cost = 4,83$.

¿Te ves capaz de resolver este problema? Y, si es así, ¿cuánto tiempo calculas que tardarás en tener el código funcionando? Piénsalo y trata de no seguir leyendo inmediatamente.

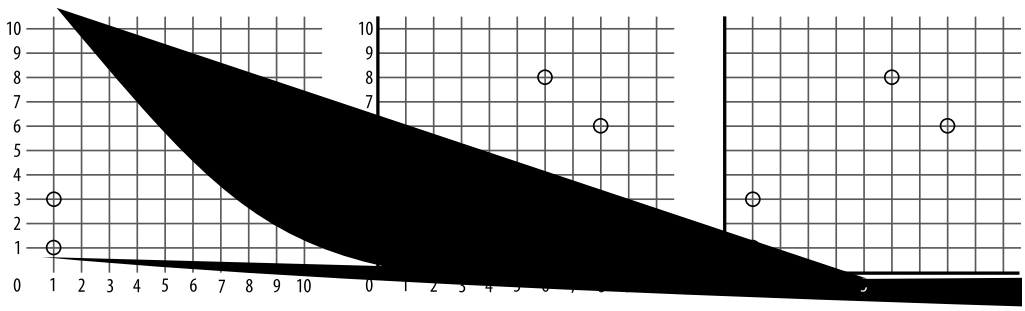



Figura 1.1: Ilustración de UVa 10911 - Forming Quiz Teams

Ahora, pregúntante con cuál de las siguientes descripciones te identifcas más. Si todavía no estás familiarizado con toda la terminología o las materias utilizadas en este capítulo, te recomendamos que lo vuelvas a leer una vez terminado el libro.

- Programador no competitivo A (alias *el difuso*):
  Paso 1: lee el problema y se queda confuso (el problema es nuevo para él).
  Paso 2: intenta programar algo a partir de las entradas y salidas de ejemplo.
  Paso 3: se da cuenta de que *ninguno* de sus intentos es **aceptado (AC)**:
  - **Voraz** (sección 3.4): emparejar repetidamente a los dos estudiantes restantes con las distancias de separación más cortas será una **respuesta incorrecta (WA)**.
  - **Búsqueda completa** ingenua: utilizar *backtracking* recursivo (sección 3.2) y probar todas las parejas posibles terminará con **tiempo límite superado (TLE)**.
- Programador no competitivo B (abandona):
  Paso 1: lee el problema y se da cuenta de que es un problema de emparejamiento, pero todavía no sabe cómo resolverlo. No conoce la solución utilizando **programación dinámica (DP)** (sección 3.5).
  Paso 2: ignora el problema y pasa al siguiente.

- Programador (todavía) no competitivo C (lento):
  Paso 1: lee el problema y se da cuenta de que es difícil **'peso mínimo que se ajuste perfectamente a un grafo de peso general'**. Sin embargo, como el tamaño de entrada es pequeño, este problema se puede resolver con DP. El estado de la DP es una **máscara de bits**, que describe el estado de emparejamiento y, al emparejarse los estudiantes desemparejados *i* y *j*, se activarán dos bits *i* y *j* en la máscara (consultar el Volumen II).
  Paso 2: programa la rutina de E/S, escribe la DP recursiva, prueba y **depura**...
  Paso 3: pasan *tres horas* y su solución logra AC (resuelve los casos de prueba secretos).
- Programador competitivo D:
  Completa todos los pasos del programador no competitivo C en ×30 minutos.
- Programador muy competitivo E:
  Un programador muy competitivo (por ejemplo, los programadores con nivel rojo en Codeforces [4]) resolvería este problema 'bien conocido' en ×10 minutos y, posiblemente, sería conocedor de las soluciones necesarias para las variantes más difíciles del mismo.

### Ejercicio 1.1.1

La estrategia voraz del programador no competitivo A, mencionado antes, funciona para el caso de ejemplo mostrado en la figura 1.1, ya que es poco habitual que los autores de los problemas incluyan casos límite en los ejemplos. Intenta hallar un contraejemplo *mejor*.

### Ejercicio 1.1.2

Analiza la complejidad de tiempo de la solución ingenua de búsqueda completa del programador no competitivo A, para entender por qué recibe un veredicto de TLE.

### Ejercicio 1.1.3*

En realidad, una solución inteligente de *backtracking* recursivo *con poda* puede resolver este problema (con $1 \times N \times 8$). Resuelve el problema *sin utilizar* una tabla de DP.

## 1.2 Los concursos

En el mundo existen varios concursos de programación. En esta sección, mencionaremos dos de los más importantes. Los autores del libro están (profundamente) involucrados en ambos.

### 1.2.1 Olimpiada internacional de informática (IOI)

#### Historia y formato

La IOI se fundó en 1989 (en Bulgaria) y se ha venido celebrando anualmente desde entonces. Singapur será el país anfitrión de las ediciones de 2020 y 2021, en las que los autores de este libro jugarán un papel fundamental. La estadísticas relativas a la IOI se pueden encontrar en https://stats.ioinformatics.org/.

Formato de la IOI: selección en el centro educativo de secundaria (opcional), selección provincial (opcional), Olimpiada informática nacional u otro sistema de selección nacional (cada país/región puede presentar a un máximo de 4 concursantes) y, finalmente, la IOI propiamente dicha (que se suele celebrar entre los meses de junio y septiembre).

#### Elegibilidad y selección

Las reglas de elegibilidad de la IOI se encuentran recogidas en su reglamento[3].

Como los participantes de la IOI pueden proceder de diferentes centros de educación secundaria de un mismo país o región, los equipos (grandes) ya formados suelen realizar pruebas de selección previas en internet, celebrar campus de entrenamiento intensivos en una ubicación centralizada y, gradualmente, depurar la composición del equipo mediante una Olimpiada nacional u otro método, hasta determinar quienes son los 4 mejores concursantes. Esto garantiza que esos 4 representantes tendrán las mejores competencias en informática, especialmente en el ámbito de la programación competitiva, de entre todos los disponibles para representar a su país o región en un año dado.

El equipo de la IOI de un país o región determinados suele contar con un líder de equipo con experiencia en la formación y selección en su ámbito local. También es habitual encontrar presencia de representantes del Ministerio de Educación o preparadores del país en cuestión.

#### Estrategias habituales para concursar

Los concursos de la IOI consisten, normalmente, en una sesión de 2 horas de prácticas[4] y dos días de concurso[4], con sesiones de 5 horas. La IOI es un concurso individual[5]. Cada concurso consta, normalmente, de 3 tareas, siendo una más sencilla, otra intermedia y una difícil, que, a su vez, están divididas en subtareas con diferente puntuación.

[3] Los problemas se suelen distribuir en línea unas semanas antes de la celebración del concurso.

[4] Es importante mantener la atención y la concentración durante *los dos días* del concurso.

[5] La IOI de 2018 utilizó, por única vez, una estadística en tiempo real de las puntuaciones, con el fn de ayudar a los concursantes a identifcar las tareas más sencillas.

El Comité científco internacional (ISC) procura asegurarse de que la suma de los puntos de las subtareas sea lo sufcientemente diverso como para minimizar la posibilidad de empate[6] (especialmente en las zonas de puntuación limítrofes con las medallas). Para facilitar la preparación de los diversos equipos nacionales, el ISC publica el temario de la IOI [16].

La velocidad de programación no suele ser un factor determinante en la IOI. Se puede enviar una solución completa (100 puntos) 4 horas y 59 minutos después de comenzada la sesión y obtener la misma puntuación que quien hubiese obtenido los 100 puntos en el minuto 30. Con ello, la IOI premia el rendimiento científco, es decir, la habilidad de resolver las subtareas más complejas frente a la velocidad en resolver las más sencillas[7].

*Históricamente*, para obtener medallas de oro, plata o bronce, será necesario obtener puntuaciones superiores a 400, 300 o 200, respectivamente, a lo largo de los dos días de competición.

El objetivo primario de este libro es lograr que el número de participantes de la IOI que obtengan menos de 70 puntos después de los días de concurso, sea el menor posible. No todas las subtareas de la IOI son difíciles, pues el ISC también procura no desmoralizar a la mitad de los concursantes, que volverán a sus casas sin ninguna medalla. Después de todo, siguen siendo futuros informáticos.

#### Lo que viene después

Muchos, *aunque no todos*, de los concursantes y medallistas de la IOI continúan sus estudios en el campo de las ciencias de la computación al llegar a la universidad[8]. También muchos, *aunque no todos*, pasan al siguiente nivel de los concursos de programación: el ICPC (que veremos en la subsección 1.2.2). Por lo tanto, quien ya conozca este libro desde su etapa de secundaria, puede utilizarlo también en la universidad.

### 1.2.2 *International Collegiate Programming Contest*

#### Historia y formato

El ICPC[9] se fundó en 1970 en los Estados Unidos, y en la década de 1990 comenzó su andadura internacional. Desde el año 2000 (con las excepciones de 2003 y 2007), los campeones han pertenecido a universidades rusas (sobre todo en el periodo 2012-2019) y asiáticas.

Formato del ICPC: selección a nivel de universidad (opcional), concurso preliminar (opcional), concurso regional (celebrado normalmente entre los meses de octubre y diciembre). Los campeones y, ocasionalmente los subcampeones y otros equipos de menor entidad, de las diferentes regiones, llegan a la fnal mundial que se celebra al año siguiente (habitualmente entre abril y julio). Los volúmenes de participación han crecido signifcativamente a lo largo de los años (desde menos de 10 000 concursantes en el año 2000, hasta más de 50 000 en 2020) [57].

[6] El ISC puede utilizar sistemas de puntuación con decimales para lograr este aspecto, como en las IOI de 2015, 2017 y 2019.

[7] Implementar soluciones para las subtareas de menor puntuación, independiente de lo rápido que pueda hacerse, consume tiempo. Por lo tanto, no es una estrategia óptima para aquellos concursantes cuyo objetivo está en las medallas más valoradas, y que deberían dedicar ese tiempo en diseñar la mejor solución posible.

[8] Muchos de ellos con becas.

[9] El ICPC estuvo ligado históricamente a la *Association for Computing Machinery* (ACM).

Como el ICPC es un concurso de programación entre universidades, los preparadores suelen pertenecer a los departamentos de informática de las mismas, siendo habitualmente profesores de programación o de algoritmia.

El ICPC tiene una duración de 5 horas. Cada equipo está formado por tres estudiantes de la misma universidad y solo dispone de un ordenador. Solo sumarán un punto aquellos envíos que resulten aceptados (completamente correctos). Se penaliza el envío de soluciones no aceptadas (normalmente se añaden 20 minutos al tiempo total). La clasifcación se forma por el número decreciente de problemas resueltos y, en caso de empate, por un menor tiempo de penalización. En el extraño caso de que el empate se mantenga, se considerará mejor que la última solución aceptada se haya enviado antes. La mayoría de los concursos fnalizan aplicando únicamente el segundo criterio de clasifcación (el campeón y el subcampeón han resuelto el mismo número de problemas, pero uno más rápido que el otro). No es habitual lograr la victoria habiendo resuelto un problema más que el segundo clasifcado. Hacerlo con dos más es extremadamente raro y se puede considerar una hazaña notable.

### Elegibilidad y selección

A diferencia de la IOI, en el que los concursantes compiten de forma individual, en representación de sus países o regiones, en el ICPC no es raro encontrar, en un mismo equipo, a miembros de diferentes nacionalidades, siempre que pertenezcan a la misma universidad y cumplan con los criterios de elegibilidad establecidos.

La universidades con más tradición en el concurso, suelen contar con mecanismos propios de entrenamiento, selección y formación de equipos, antes de determinar quienes serán los mejores representantes en las fases regionales.

### Estrategias habituales para concursar

Los conjuntos de problemas del ICPC suelen estar diseñados de forma que todos los equipos puedan resolver, al menos, uno de ellos (para tratar de no desmoralizar completamente a los recién llegados al concurso, a los que tratamos de ayudar con este libro), que ningún equipo pueda resolver todos los problemas (para mantener el interés de la competición hasta el último momento) y que cada problema sea resuelto por, al menos, un equipo (lo que minimiza el número de problemas *imposibles*, que necesitarían una dedicación superior a las 5 horas incluso para los equipos mejor preparados).

Podemos dividir las cinco horas de concurso en tres fases: el inicio, el desarrollo y el desenlace. En el ICPC, la penalización de tiempo juega un papel fundamental, por lo que la capacidad de identifcar los problemas más fáciles, enterrados en el conjunto de 10-13 problemas, y resolverlos lo más rápidamente posible logrando, con ello, una penalización mínima, resulta crucial. En algunos concursos (aunque no es una norma escrita) el problema más sencillo es el A (la primera página). En el caso de los mejores equipos, el rendimiento mostrado al inicio determina el tono del resto del concurso, es decir, liderarlo o ir a remolque de otros. Durante esta fase, la mayoría de los equipos funcionan en *modo individual*, donde cada uno de los tres concursantes trata de resolver uno de los 3 problemas más fáciles por separado. Los equipos que dependen de la clasifcación para identifcar las cuestiones más sencillas siempre se verán demorados.

[1] Algunos antiguos concursantes de la IOI tendrán que mejorar su velocidad de implementación para el ICPC.

El desarrollo suele comenzar entre las segunda y tercera horas. Para entonces, los tres miembros del equipo deberían haber leído todos los problemas (que todavía no hayan resuelto) y clasificarlos en base a su dificultad (aparente), tras consultar la clasificación en ese momento. En el caso de los equipos punteros, la capacidad de seguir produciendo resultados durante esta fase es imprescindible. No se debe llegar a un punto en el que haya incertidumbre sobre cuál debe ser el siguiente problema a resolver o implementar. Como, en este momento, nos encontraremos ante problemas de dificultad media-alta (al menos desde la subjetividad de los concursantes), el trabajo en equipo es importante. Aquellos equipos extraordinarios, con tres concursantes sólidos, podrían seguir trabajando de forma individual. Otros equipos preferirán una combinación de dos concursantes más uno. Y el resto, seguramente, tendrán que sumar las fuerzas de los tres miembros para lograr resolver un problema más.

El desenlace comienza cuando el equipo ya ha resuelto todos los problemas fáciles y medios, y se encuentra con aquellos que no ha resuelto antes, o que le han ocupado mucho tiempo durante las sesiones de preparación. A los mejores equipos les quedarán solo unos pocos problemas y deberían ser capaces de estimar cuánto tiempo necesitan para resolver uno más y enviar un código aceptado en el minuto 299, en vez de hacerlo pasado el 300. Para la mayoría del resto de equipos, esta fase supondrá un intento de mejorar el resultado, con sus 3 miembros trabajando intensamente en un solo problema, con la esperanza de no bloquearse. La mayoría de los concursos finalizan sin un resultado claro, en el que el equipo campeón haya resuelto todos los problemas, pues los jueces tratan de que el tiempo teórico necesario para resolver todo el conjunto de problemas sea, en el mejor de los casos, superior a 5 horas.

#### Lo que viene después

La mayoría de los programadores competitivos finalizarán su etapa competitiva después de su última participación en el ICPC. Un buen rendimiento en el ICPC, durante la etapa universitaria, será, probablemente, un factor importante para destacar en las entrevistas técnicas de las principales empresas de tecnologías de la información.

### 1.2.3 Otros concursos de programación

Finalizado el periodo universitario, el concursante podrá encontrar otros concursos de programación, la mayoría de ellos en línea (y, quizá, con una final presencial). Algunos de ellos son *Google CodeJam*, *Facebook Hacker Cup*, *Topcoder Open*, concurso de *Codeforces*, *Internet Problem Solving Contest* (IPSC), etc. Son competiciones que no están tratadas específicamente en el presente libro.

## 1.3 Consejos para ser competitivo

Si quieres convertirte en los programadores competitivos D o E, mencionados en la sección 1.1, es decir, si quieres llegar a participar (a través de las selecciones nacionales) y lograr una medalla en la IOI [1], ser uno de los miembros de los equipos que representen a tu universidad

en el ICPC [57] (finales nacionales, regionales, mundiales) u obtener buenos resultados en otros concursos de programación, no hay duda de que este libro[11] es para tí.

En los próximos capítulos, aprenderás todo lo necesario desde el nivel básico al intermedio o, incluso, avanzado[12], sobre estructuras de datos y algoritmos que han aparecido en concursos de programación recientes, recopilados de diversas fuentes [42, 7, 50, 38, 51, 40, 53, 1, 35, 6, 52, 39, 5, 54, 43, 18, 37, 36] (ver la figura 1.4). No solo conocerás los conceptos que están detrás de las estructuras de datos y los algoritmos, sino que sabrás implementarlos eficientemente y aplicarlos a los problemas de programación apropiados. Además, descubrirás muchos consejos sobre programación, derivados de nuestra propia experiencia, que pueden resultar útiles en un concurso. Vamos a comenzar con algunos consejos generales.

### 1.3.1 Consejo 1: escribe más rápido

No es broma. Aunque este consejo puede no tener mucho sentido, dado que ni el ICPC ni (especialmente) la IOI son concursos de mecanografía, hemos visto equipos en el ICPC clasificados en las posiciones $i$ e $i + 1$ separados por unos pocos minutos [13], así como concursantes de la IOI frustrados al perder puntos importantes por no ser capaces de programar adecuadamente una solución de fuerza bruta en el último minuto. Cuando eres capaz de resolver el mismo número de problemas que tu contrincante, la diferencia vendrá dada por la capacidad de producir código preciso y robusto y por... la velocidad al escribirlo.

Puedes conocer tu velocidad con el teclado en `http://www.typingtest.com`, y seguir las instrucciones mostradas para mejorar tus capacidades en ese sentido. Steven logra 85-95 palabras por minuto[14], Felix está en 55-65 y Suhendry en 70-80. Si tu velocidad es muy inferior a estos números, deberías tomarte este consejo muy en serio.

Además de ser capaz de escribir caracteres alfanuméricos rápidamente y de forma correcta, deberás familiarizarte también con las posiciones de las teclas utilizadas más frecuentemente por los lenguajes de programación: paréntesis `()`, llaves `{}`, corchetes `[]` o los signos `<>`; el punto y coma `;` o los dos puntos `:`, las comillas simples `'` para los caracteres, las comillas dobles `"` para las cadenas, el *ampersand* `&`, la barra vertical o 'tubería' `|`, el signo de exclamación `!`, etc.

A modo de práctica, teclea el siguiente código C++ lo más rápidamente posible[15]:

[11] No hay que olvidar que en los concursos más importantes, solo puede haber un ganador, es decir, la probabilidad de no ganar nunca durante tu etapa de programador competitivo es mucho más alta que la contraria. Sin embargo, aunque es importante mantener expectativas altas y tratar de ganar, al menos, un concurso de programación, el objetivo último debería ser mejorar la capacidad de programación y de resolución de problemas.

[12] La percepción del material presentado en este libro como intermedio o avanzado depende de los conocimientos previos del lector en lo relativo a programación, algoritmia y capacidad de resolución de problemas.

[13] Un rendimiento rápido en la fase inicial del ICPC puede ser muy beneficioso. Como ilustración, los equipos A y B resuelven un total de 8 problemas. El equipo A logra su primer AC 5 minutos antes que el equipo B. Después, resuelven los 7 siguientes problemas a la misma velocidad. El equipo A ganará al obtener una penalización de $5 + 8 \cdot 20 = 40$ minutos menos.

[14] Algunos de los alumnos del autor, que han llegado a finalistas mundiales en el ICPC, son capaces de superar las 120 palabras por minuto. La media en la población general se encuentra en las 40 palabras por minutos.

[15] Vemos que el estilo típico de la programación competitiva incumple muchas buenas prácticas de los principios de la ingeniería de software, utilizando variables globales, nombres de variables extremadamente cortos y crípticos, inclusión de todas las cabeceras disponibles, excesivo uso de la manipulación de bits, la sentencia `using namespace std`, etc.

```
          'PSNJOH 2VJ[ 5FBNT  TPMVDJ³O BM QSPCMFNB 67B
 JODMVEF  CJUT TUED  I                  UPEBT MBT CJCMJPUFDBT
VTJOH OBNFTQBDF TUE

 EFGJOF -40OF 4   4     4               BDFMFSBDJ³O JNQPSUBOUF

JOU/                                    N¡Y /
EPVCMEEJTU<  ><  >  NFNP<     >                 ?

EPVCMEEQ •?& B •$0Ð
```

### 1.3.2 Consejo 2: identifica rápidamente el tipo de problema

En los ICPC recientes, los concursantes (equipos) reciben un **conjunto** de ^ 10-13 problemas de diferentes tipos. De nuestra observación de esos conjuntos de problemas, en concursos regionales de Asia y en las fnales mundiales, podemos categorizar los tipos de problemas y su frecuencia de aparición en la siguiente tabla[1]. En la IOI, los concursantes reciben 6 tareas que deberán resolver a lo largo de dos días[1], que cubre los elementos 1-7 y parte del 11, con un subconjunto *muy reducido* de los elementos 8-11. Par más información, recomendamos consultar la clasifcación de problemas de las IOI 1989-2008 [58] y el temario más reciente de la IOI [16].

| Nº | Categoría | En este libro | Frecuencia |
|---|---|---|---|
| 1. | *Ad hoc* | Secciones 1.4-1.6 | 1-2 |
| 2. | Estructuras de datos (pesadas) | Capítulo 2 | 0-1 |
| 3. | Búsqueda completa (iterativa/recursiva) | Sección 3.2 | 1-2 |
| 4. | Divide y vencerás | Sección 3.3 | 0-1 |
| 5. | Voraz (no clásicos) | Sección 3.4 | 1 |
| 6. | Programación dinámica (no clásicos) | Sección 3.5 | 1-2 |
| 7. | Grafos (salvo flujo de red/emparejamiento de grafos) | Capítulo 4 | 1 |
| 8. | Matemáticas | Capítulo 5 | 1-2 |
| 9. | Procesamiento de cadenas | Capítulo 6 | 1 |
| 10. | Geometría computacional | Capítulo 7 | 1 |
| 11. | Algunos problemas difíciles/poco habituales/novedosos | Capítulos 8-9 | 2-3 |
| | **Total en el conjunto normalmente** X 14 | | **10-17** |

Tabla 1.1:Tipos de problemas en los ICPC regionales de Asia recientes

La clasifcación de la tabla 1.1 está adaptada de [44] y no pretende ser exhaustiva. Algunas técnicas, como la ordenación, no aparecen aquí, ya que se consideran triviales y, normalmente, serán utilizadas como subrutinas de un problema más amplio. Tampoco hemos incluido la recursividad, ya que está implícita en categorías como *backtracking* recursivo o la programación dinámica. Por supuesto, en ocasiones, los problemas requieren técnicas mixtas: un problema puede estar clasifcado en más de un tipo. Por ejemplo, el algoritmo de Floyd–Warshall es tanto una solución para problemas de grafos, como el de los caminos más cortos entre todos los pares (APSP, sección 4.5), como un algoritmo de programación dinámica (sección 3.5). Los algoritmos de Prim y Kruskal son, igualmente, soluciones para problemas de árbol de expansión mínimo (MST, sección 4.3) y algoritmos voraces (sección 3.4). En el Volumen II, veremos problemas (más difíciles) para cuya solución son necesarios más de un algoritmo y/o estructura de datos.

| Nº | Categoría | Confianza y expectativa de resolución |
|---|---|---|
| A1. | Ya he resuelto este tipo | Estoy seguro de resolverlo otra vez (y rápido) |
| A2. | Ya he resuelto este tipo | Estoy seguro de resolverlo otra vez (pero lento) |
| B. | Ya he visto este tipo | Pero no supe resolverlo |
| C. | No he visto este tipo | Seguir leyendo |

Tabla 1.2:Tipos de problemas (versión compacta)

En un futuro (próximo) estas clasifcaciones podrían cambiar. Un ejemplo signifcativo es la pro-

[1] En los años 2009-2010, la IOI contaba con 8 tareas repartidas en dos días con, al menos, una muy sencilla cada día. Sin embargo, este formato se abandonó en favor de una subtarea muy sencilla en cada una de las tareas.

gramación dinámica. Esta técnica, desconocida antes de la década de 1940, y poco utilizada en el ICPC o la IOI antes de mediados de la década de 1990, es considerada un requisito fundamental hoy día. Para ilustrarlo, sirva que en las finales mundiales del ICPC de 2010 hubo 3 y 3 problemas de DP (de un total de 11).

Sin embargo, el objetivo principal *no es* simplemente asociar problemas con las técnicas necesarias para resolverlos, como en la tabla 1-1. Una vez que conozcas la mayoría de los temas de este libro, deberías de ser capaz de catalogar los problemas en los cuatro tipos de la tabla 1-2.

Para ser *competitivo*, es decir, obtener *un buen resultado* en un concurso de programación, deberás ser capaz de clasificar problemas frecuentemente y con confianza en el tipo A1, y minimizar el número de problemas que quedan en los tipos A2 o B. En suma, debes adquirir los suficientes conocimientos de algoritmia y desarrollar tus capacidades de programación como para considerar sencillos la mayoría de los problemas clásicos, sobre todo en el inicio del concurso.

Sin embargo, para *ganar* un concurso de programación, deberás tener *capacidad de resolución de problemas* avanzada, para que tú (o tu equipo) seas capaz de reconducir la solución requerida para un problema difícil/original del tipo C en la IOI o el ICPC, y hacerlo *durante* el concurso, no *después* de que el autor del problema, o el juez correspondiente, hagan pública la solución. Algunas de las capacidades de resolución de problemas necesarias son:

- Reducir el problema dado a otro más sencillo.
- Reducir un problema (NP-)complejo al problema dado.
- Identificar las pistas sutiles o las propiedades especiales del problema.
- Atacar el problema desde un ángulo poco evidente, o hacer la pregunta de otra forma.
- Comprimir los datos de entrada.
- Reescribir fórmulas matemáticas.
- Hallar patrones en los casos de prueba.
- Realizar análisis de casos de los posibles subcasos del problema.

### Ejercicio 1.3.2.1

| UVa/Kattis | Título | Tipo de problema | Más información |
|---|---|---|---|
| wordcloud | Word Cloud | | Sección 1.6 |
| turbo | Turbo | | Sección 2.4 |
| 10360 | Rat Attack | Búsqueda completa o DP | Sección 3.2 |
| hindex | H-Index | | Sección 3.3 |
| 11292 | Dragon of Loowater | | Sección 3.4 |
| 11450 | Wedding Shopping | | Sección 3.5 |
| 11512 | GATTACA | | Volumen II |
| 10065 | Useless Tile Packers | | Volumen II |
| 11506 | Angry Programmer | | Volumen II |
| bilateral | Bilateral Projects | | Volumen II |
| carpool | Carpool | | Volumen II |

Tabla 1.3: Ejercicio: leer y clasifcar estos problemas de UVa/Kattis

Lee los problemas de UVa [45] y Kattis [34] de la tabla 1.3 y determina sus tipos. Uno de ellos ya aparece identifcado. Completar esta tabla será sencillo después de leer este libro, ya que todas las técnicas necesarias para resolver los problemas aparecen aquí descritas.

### Ejercicio 1.3.2.2*

Utilizando la misma lista de problemas de la tabla 1.3, resume los mismos en un máximo de tres frases, omitiendo los detalles o narrativa irrelevantes, pero conservando los puntos clave de los problemas, de forma que otro programador competitivo que *no haya leído* los enunciados originales pueda escribir soluciones correctas para ellos. En la primera página del libro encontrarás un ejemplo de ello.

## 1.3.3 Consejo 3: analiza el algoritmo

Una vez que hayas diseñado un algoritmo que resuelva un problema en particular de un concurso de programación, debes hacerte la siguiente pregunta: dado el límite superior máximo del tamaño de la entrada (indicado normalmente en el enunciado del problema, si está bien redactado), ¿es el algoritmo actual capaz de, con su complejidad de espacio/tiempo, resolver el problema dentro de los límites de tiempo y memoria indicados?

En ocasiones, hay más de una forma de atacar un problema. Algunos de los abordajes pueden ser incorrectos, otros no lo sufcientemente rápidos y otros 'excesivos'. Una buena estrategia consiste en plantear los diferentes algoritmos y elegir **la solución más sencilla que funcione** (es decir, es lo bastante rápida para cumplir con los límites de tiempo y memoria y devuelve la respuesta correcta)[17].

[17] Es cierto que, en los concursos de programación, elegir el algoritmo más sencillo que funciona es crucial para un

Los ordenadores modernos son muy rápidos y pueden procesar[18] hasta $\sim 100M$ (o $10^8$, $1M = 1\,000\,000$) operaciones por segundo. Esta información es útil para determinar si el algoritmo actual cumplirá con el límite de tiempo. Por ejemplo, si el tamaño máximo de entrada es $100K$ (o $10^5$, $1K = 1000$), y el algoritmo en cuestión tiene una complejidad de $O(n^2)$, el sentido común (o una calculadora) indicará que $(100K)^2$ o $10^{10}$, es un número muy grande para el que el algoritmo necesitará (del orden de) cientos de segundos de ejecución. Por lo tanto, será necesario plantear un algoritmo más rápido (y también correcto) que resuelva el problema. Vamos a suponer que encontramos uno cuya complejidad sea de $O(n \log_2 n)$. En este caso, la calculadora nos informará de que $10^5 \log_2 10^5$ es solo $1{,}7 \times 10^6$ y el sentido común nos dictará que el algoritmo (que ahora se ejecutará en menos de un segundo) probablemente estará dentro del límite de tiempo.

Los límites del problema son tan importantes[19] como la complejidad del algoritmo, para determinar si la solución es adecuada. Supongamos que solo podemos plantear un algoritmo relativamente simple de implementar que se ejecuta con la horrorosa complejidad de $O(n^4)$. De entrada, puede parecer una solución inaceptable, pero si $n \leq 50$, habremos resuelto el problema. Podemos implementar el algoritmo de $O(n^4)$ impunemente, ya que $50^4$ resulta ser 6,25M, y eso únicamente ocupará alrededor de un segundo de tiempo de ejecución.

Hay que tener en cuenta, sin embargo, que el orden de complejidad no indica, necesariamente, el número real de operaciones que requerirá el algoritmo. Si cada iteración implica un número importante de operaciones (ya sean de coma fotante o con un número signifcativo de bucles anidados constantes), o si la implementación tiene un consumo 'constante' alto en su ejecución (bucles repetidos innecesariamente, pasadas múltiples sobre el conjunto de datos o, incluso, sobrecargas en la ejecución de la E/S), el código puede tardar en ejecutarse más de lo esperado. De todas formas, esto no suele ser un problema grave, ya que los autores de los problemas deberían haber diseñado los límites de tiempo de forma que las implementaciones más razonables del algoritmo, en relación a la complejidad de tiempo, logren un veredicto de aceptado (AC).

Al analizar la complejidad del algoritmo con los límites de entrada, tiempo y memoria dados, se puede decidir con más facilidad si se debería intentar implementar directamente (lo que ocupa un tiempo precioso en las IOI y los ICPC), intentar mejorarlo primero o, en último caso, ocuparse de un problema diferente.

Como se ha mencionado en el prefacio de este libro, vamos a intentar *no* tratar el concepto del análisis de algoritmos en detalle. *Asumimos* que el lector ya cuenta con esas habilidades básicas. Hay muchos libros de referencia (por ejemplo *"Introduction to Algorithms"* [5], *"Algorithm Design"* [35], *"Algorithms"* [6], etc.) que ayudarán a entender los siguientes conceptos y técnicas necesarios para el análisis de algoritmos:

- Análisis de complejidad de tiempo y espacio básico, para algoritmos iterativos y recursivos:
    - Un algoritmo con $k$ bucles anidados de unas $n$ iteraciones cada uno, tiene una complejidad de $O(n^k)$.

buen resultado. Sin embargo, durante las *sesiones de entrenamiento*, donde no tenemos limitaciones de tiempo, puede ser benefcioso dedicar más esfuerzos a resolver un problema determinado utilizando *el mejor algoritmo posible*. De esta forma, si en el futuro encontramos una versión más compleja del problema, tendremos más posibilidades de obtener e implementar la solución correcta.

[18] Hay que tomarse esto como una aproximación, ya que el número puede variar entre diferentes equipos y mejorará con el paso del tiempo. Un programador competitivo debería verifcar estos números durante la sesión de práctica.

[19] Si eres autor de problemas, por favor presta mucha atención a esos límites.

- “ 4J FM BMHPSJUNP FTCSMFDBVSBTEJBWTPSEDPVOSTJWBTQJPVSFOUFM Z
  UFOESŸ VOB DPNQMFKJEBE BQSPYJNBEBMFNJUF TVQFSJPS FT
  SJBCMF -B DPNQMFKJEBE SFBM WFOESŸ EBEB QPS MBT BDDJP
  Z EFQFOEJFOEP EF TJ FT QPTJCMF SFBMJ[BS QPEB
- “ 6O BMHPSJUNP EF QSPHSBNBDJ±O EJOŸNJDB V PUSB SVUJO
  NBUSJ[ CJEJNFOTJPOBM QBSB DFMEB UJFOF VO UJFNQP EF FKFDVD
  &TUF BTQFDUP FTUŸ NFKPS FYQMJDBEP FO MB TFDDJ±O
- “ -B C¸TRVFEB CJOBSJB FOCONTUSUYEN EL PNUNQMFMEKJEBE EF

- 5§DOJDBT EF BOŸMJTJT NŸT BWBO[BEBT

  - “ 1SPCBS MB WBMJEF[ EF VO BMHPSJUNP FTQFDJBMNFOUF MPT
    QBSB NJOJNJ[BS MB QPTJCJMJEBE EF PCUFOFS VO WFSFE
  - “ 3FBMJ[BS VO BOŸMJTJT BNPSUJ[BEP WFS QPSBMKHPNRQWFP FM
    SBSBNFOUF TF IBDF FO MPT DPODVSTPT QBSB NJOJNJ[BS FM S
    EF UJFNQP M«NJUF TVQFSBEP P QFPS B¸O DPOTJEFSBS FM
    JHOPSBS FM QSPCMFNB DVBOEP FO SFBMJEBE FT MP TVSDJF
    EFSBEB MB BNPSUJ[BDJ±O
  - “ 3FBMJ[BS VO BOŸMJTJT DVJEBEPTP EF MB TBMJEB SFRVFSJEE
    UBNCJ§O EFQFOEF EFM UBNB¯P EF MB TBMJEB Z NJOJNJ[BS I
    VO WFSFEJDUP EF UJFNQP M«NJUF TVQFSBEP 1PS FKFNQMP
    BMHPSQJBUSUPBM@FTTDSJMLH &M UJFNQP FNQMFBEP FO MB FKFDVD
    SJUNP OP EFQFOEF TPOEF MFBMFUOBNSB¯EB TJOP UBNCJ§O EFM U
    TBMJEBO¸MPFTSPT NŸT QFRVF¯PT P NŸT HSBOEFT RVF TF EFCF
    NŸT EFUBMMFT FO MB TFDDJ±O

- 'BNJMJBSJEBE DPO MPT TJHVJFOUFT M«NJUFT

  - “ ^ ^
  - “ -PT FOUFSPT DPOJTUJHZOMEFH MPCJHUT UJFOFO VOPT M«NJUFT TV
    EF ( ^ — TF QVFEFO VUJMJ[BS DPO^TFE«VHSJUEBTEEBDUBBDMFOT
    Z ( ^ — IBTÚB E«HJUPT EFDJNBMFT SFTQFDUJWBNFO
  - “ -PT FOUFSPT TJO TJHOP TF VUJMJ[BSŸO ¸OJDBNFOUF DVBOE
    HBUJWPT -PT FOUFSPTTOTJHOPTJEHZOPUEFTJHOFE MPOHCMHTH
    UJFOFO VOPT M«NJUFT(TV^QF–SJPZSFT( EF^ —
  - “ 4J FT OFDFTBSJP BMNYBDFTOFBESFFOFUVFUSJMJ[BGTHFFDSDJ±O
  - “ &YJTOFQOFSNVUBDJPVCEZOKVOUFMTFNBSOPT EJTUJOUPT
  - “

4. Dado un conjunto múltiple $S$ de $M = 100K$ enteros, queremos saber cuántos enteros diferentes podemos formar si elegimos dos cualesquiera de $S$ (no necesariamente distintos) y los sumamos. El contenido del conjunto múltiple $S$ está formado por números primos no mayores que $20K$. ¿Qué algoritmo no superaría $100M$ de operaciones en el peor de los casos?

   *a*) Comprobar los $O(M^2)$ pares de enteros posibles e insertar sus sumas en una tabla de *hash* ($O(1)$ por inserción). Terminar informando del tamaño de la tabla de *hash*.

   *b*) Ejecutar un algoritmo como el anterior, pero después de aplicar la técnica: _____ .

5. Debes calcular el camino más corto entre dos vértices de un grafo acíclico dirigido ponderado (DAG) con $|V|, |E| \leq 100K$. ¿Qué algoritmos se pueden utilizar?

   *a*) Programación dinámica (secciones 3.5, 4.2.6 y 4.6.1).

   *b*) Búsqueda en anchura (secciones 4.2.3 y 4.4.2).

   *c*) Dijkstra (sección 4.4.3).

   *d*) Bellman–Ford (sección 4.4.4).

   *e*) Floyd–Warshall (sección 4.5).

6. ¿Qué algoritmo genera una lista de los $10K$ primeros números primos con la mejor complejidad de tiempo? (Volumen II)

   *a*) Criba de Eratóstenes.

   *b*) $\forall$`i` $\in [1..10M]$, si `isPrime(i)` es verdadero, añadir `i` a la lista.

7. ¿Cómo comprobar si el factorial de $n$, es decir, $n!$, es divisible por un entero $m$? $1 \leq n \leq 10^{14}$.

   *a*) Comprobar si $n!\ \%\ m = 0$.

   *b*) La técnica ingenua no sirve, utilizar: _____ (Volumen II).

8. Quieres enumerar todas las apariciones de una subcadena $P$ (de longitud $m$) en una cadena (larga) $T$ (de longitud $n$), si las hay. Tanto $n$ como $m$ tienen un máximo de $1M$ caracteres. ¿Qué algoritmo es más rápido?

   *a*) Utilizar el siguiente código de C++:

```
for (int i = 0; i < n-m; ++i) {
  bool found = true;
  for (int j = 0; (j < m) && found; ++j)
    if ((i+j >= n) || (P[j] != T[i+j]))
      found = false;
  if (found)
    printf("P is found at index %d in T\n", i);
}
```

*b*) Hay algoritmos mejores, podemos utilizar: _____ (Volumen II).

9. Dado un conjunto $S$ de $N$ puntos distribuidos en un plano bidimensional ($2 \leq N \leq 5000$), hallar dos puntos $\in S$ separados por la mayor distancia euclídea. ¿Es posible utilizar un algoritmo de búsqueda completa en $O(N^2)$ que compruebe todos los pares?

   *a*) Sí, esa búsqueda completa es posible.

   *b*) No, debemos utilizar otro método como: _____ .

10. Repitamos la pregunta anterior, pero ahora con un conjunto de puntos más grande, $2 \leq N \leq 200K$, y una restricción añadida: los puntos están *distribuidos aleatoriamente* en el plano bidimensional.

    *a*) Todavía se puede utilizar la búsqueda completa en $O(N^2)$.

    *b*) Esa técnica ingenua no servirá, utilizar: _____ (Volumen II).

11. Seguimos con la pregunta anterior. Se mantiene el conjunto de $2 \leq N \leq 200K$ puntos. Pero, esta vez, la *distribución aleatoria* de los puntos en el plano bidimensional *no está garantizada*.

    *a*) Todavía se puede utilizar la búsqueda completa en $O(N^2)$.

    *b*) Todavía se puede utilizar la solución mejor del algoritmo del Volumen II.

    *c*) Debemos utilizar: _____ .

### 1.3.4 Consejo 4: domina los lenguajes de programación

En el ICPC se permite utilizar varios lenguajes de programación[25], incluyendo C/C++, Java y Python. ¿Qué lenguajes de programación se deben intentar dominar?

Nuestra experiencia nos da esta respuesta: preferimos C++ (`std=gnu++17`) acompañado de la *Standard Template Library* (STL) incluida, pero, además, es necesario tener conocimientos avanzados de Java y algunas nociones de Python. Aunque resulta más lento, Java incluye algunas bibliotecas y APIs muy potentes, como `BigInteger/BigDecimal`, `GregorianCalendar`, `Regex`, etc. Los programas en Java son más sencillos de depurar, con la capacidad de la máquina virtual de proporcionar un trazado de la pila cuando se produce un error de ejecución (a diferencia de los volcados o errores de segmentación de C/C++). Igualmente, el código de Python puede ser sorprendentemente corto para algunas tareas concretas. Por otro lado, C/C++ también cuenta con méritos propios. Dependiendo del problema que nos ocupe, la mejor opción puede ser uno u otro lenguaje a la hora de implementar una solución en el menor tiempo posible.

[25]Opinión personal: según el reglamento de la IOI de 2019, los lenguajes admitidos son C++ y Java (la Asamblea General ha decidido en 2020 abandonar Java y otros dos lenguajes, Pascal y C, han sido retirados recientemente). La final mundial del ICPC (y, con ello, la mayoría de las regionales) permite el uso de C, C++, Java y Python (parcialmente) durante el concurso. Por ello, parece claro que el 'mejor' lenguaje a dominar en 2020 sigue siendo C++ (`std=gnu++17`), ya que está permitido en ambas competiciones, es un lenguaje rápido y cuenta con las importantes aportaciones de la STL. Si los concursantes de la IOI deciden utilizar C++, tendrán la ventaja de poder utilizar el mismo lenguaje (aunque a un nivel superior) en el ámbito del ICPC. En la actualidad, OCaml no se utiliza ni en la IOI ni en el ICPC.

Vamos a suponer que un problema requiere el cálculo de 40! (el factorial de 40). La respuesta es muy grande: 815 915 283 247 897 734 345 611 269 596 115 894 272 000 000 000. Este número supera por mucho el tamaño del entero más grande de los tipos de datos incluidos en el lenguaje (`unsigned long long`: $2^{64}-1$). Como C/C++ todavía no incluye una biblioteca aritmética de precisión arbitraria, deberemos crear una nosotros mismos. El código en Python, sin embargo, es extremadamente breve:

```
import math
print(math.factorial(40))                          # todo integrado
```

El código en Java también es muy sencillo (hay más detalles en la sección 2.2.4):

```
import java.util.Scanner;
import java.math.BigInteger;

class Main {                                      // nombre estándar de clase
  public static void main(String[] args) {
    BigInteger fac = BigInteger.ONE;
    for (int i = 2; i <= 40; i++)
      fac = fac.multiply(BigInteger.valueOf(i));   // está en la biblioteca
    System.out.println(fac);
  }
}
```

También es importante dominar y entender toda la capacidad del lenguaje de programación de preferencia. Veamos este problema con un formato de entrada no estándar: la primera línea de la entrada es un entero *N*. Le siguen *N* líneas, cada una de las cuales comienza con el carácter '0', seguido de un punto '.', y ello seguido de un número desconocido (de hasta 100 dígitos), finalmente terminado con tres puntos '. . .'. La tarea consiste en extraer los dígitos.

```
3
0.1227...
0.517611738...
0.7341231223444344389923899277...
```

Una posible solución sería:

```
#include <bits/stdc++.h>                           // incluir todo
using namespace std;
int main() {
  int N; scanf("%d\n", &N);
  while (N--) {                                    // bucle de N,N-1,...,0
    char x[110];                                   // tamaño un poco más grande
    scanf("0.%[0-9]...\n", &x);                    // '&' aquí es opcional
    // nota: si te sorprende la técnica utilizada,
    // consulta el uso de scanf en https://en.cppreference.com/w/
```

```
    printf("the digits are 0.%s\n", x);
  }
  return 0;
}
```

Muchos programadores de C/C++ no son conscientes de la potencia que ofrece el uso de expresiones regulares incluido en la biblioteca de entrada/salida estándar de C. Aunque `scanf/printf` son rutinas de E/S propias de C, se pueden utilizar en el código escrito en C++. Muchos programadores de C++ se *obligan* a sí mismos a utilizar `cin/cout` constantemente, a pesar de que no son tan flexibles como `scanf/printf`, y mucho más lentas[26].

Un ejemplo sencillo más. Tenemos una matriz bidimensional. La tarea consiste en transponer dicha matriz y mostrar el resultado. Por ejemplo, digamos que la matriz es $A = [(1, 2, 3), (4, 5, 6)]$, es decir, tiene unas dimensiones de $2 \times 3$. Para esta entrada, la salida será $A' = [(1, 4), (2, 5), (3, 6)]$, lo que consiste en una matriz transpuesta de tamaño $3 \times 2$. Si estás pensando en escribir una solución basada en bucles (`for` anidados), es porque quizá no conozcas esta elegante solución en Python:

```
A = [(1, 2, 3), (4, 5, 6)]                    # lista A = 2 tuplas de 3
[*zip(*A)]                                    # [(1, 4), (2, 5), (3, 6)]
```

En los concursos de programación, sobre todo en los ICPC, el tiempo empleado en la programación *no* debería ser el cuello de botella principal. Una vez que tengas claro cuál es el 'peor algoritmo AC', que se ejecutará dentro del tiempo límite estipulado, lo normal es que lo traslades rápidamente a un código libre de errores.

| | |
|---|---|
| **C++** | `ch1/scanf.cpp` |
| **Java** | `ch1/factorial.java` |
| **Python** | `ch1/factorial.py` \| `ch1/zip.py` |
| **OCaml** | `ch1/scanf.ml` |

Ahora, intenta resolver alguno de los siguientes ejercicios. Si necesitas más de 15 líneas de código para cualquiera de ellos (compara tus respuestas con las soluciones de referencia de la sección 1.7), deberías replantearte tu conocimiento de lenguajes de programación. Dominar los lenguajes que utilizas y los recursos que ofrecen es extraordinariamente importante y te ayudará en gran medida en los concursos de programación.

### Ejercicio 1.3.4.1

Escribe código funcional, *lo más conciso posible*, para realizar las siguientes tareas. Salvo que esté indicado expresamente, puedes utilizar el lenguaje que prefieras.

1. Usando **Java**, lee un `double` (por ejemplo, `1.4732`, `15.324547327`, etc.) y escríbelo, pero con una anchura de campo mínima de 7 caracteres y con 3 dígitos después del punto decimal (por ejemplo, `ss1.473`, donde 's' indica un espacio, `s15.325`, etc.).

[26] Es posible utilizar `ios::sync_with_stdio(false); cin.tie(NULL);` para evitar la sincronización constante. De esta forma `cin/cout` serán más rápidas, aunque sin llegar a la velocidad de `scanf/printf`.

2. Utiliza `__builtin_ctz(v)` para contar el número de ceros a la derecha de un entero con signo de 64 bits `long long v`.

3. Utiliza `ms.erase(v)` para eliminar *una sola* copia del valor `v` de un `multiset<int> ms` que contenga 0, 1 o más copias de `v`.

4. Asume que `v` es un `vector<int>` que contiene varios enteros aleatorios.

```
for (int i = 1; i <= 4; ++i) v.push_back(i); // o cambiando 4 a 5
vector<int>::iterator it = v.begin();
cout << *it << "\n"; // debe escribir v[0] = 1
v.push_back(rand()); // aumenta el tamaño del vector en 1
cout << *it << "\n"; // ¿no debería v[0] seguir siendo 1?
```

5. Error sutil similar al anterior.

```
for (int i = 1; i <= 4; ++i) v.push_back(i); // o cambiando 4 a 5
auto &front = v[0];    // pasa por referencia
cout << front << "\n"; // debe escribir v[0] = 1
v.push_back(rand());   // aumenta el tamaño del vector en 1
cout << front << "\n"; // ¿no debería v[0] seguir siendo 1?
```

### 1.3.6 Consejo 6: práctica y más práctica

Los programadores competitivos, al igual que los auténticos atletas, deben entrenar con regularidad para mantenerse 'en forma'. Para este penúltimo consejo, proporcionaremos una lista de varias páginas web, con recursos que te ayudarán a mejorar tu capacidad de resolución de problemas. Creemos firmemente que el éxito viene de un esfuerzo de mejora continuo.

El Online Judge de la Universidad de Valladolid (UVa, en España)[29] (`https://onlinejudge.org`) [45], contiene problemas de ediciones pasadas del ICPC (locales, regionales y de las finales mundiales), además de problemas de otras fuentes, incluyendo varios provenientes de concursos alojados en la propia página. Puedes resolver los problemas y enviar las soluciones al Online Judge. El sistema evaluará tu programa, lo antes posible, y te entregará un veredicto. Trata de resolver los problemas mencionados en este libro y, quizá, algún día veas tu nombre en la lista de los 500 usuarios que más problemas han resuelto.

A fecha de 19 de julio de 2020, hay que resolver $\geq$ 699 problemas para estar entre los 500 mejores usuarios. Steven está clasificado en el puesto 39 (habiendo resuelto 2074 problemas), Felix está en el 72 (con 1550 problemas) y Suhendry ocupa el 124 (con 1262 problemas) de un total de $\approx$ 365 857 usuarios en el juez UVa (y un total de $\approx$ 4965 problemas). Los tres nos encontramos en el percentil 99,9.

[29] Este juez en línea dejó de estar afiliado a la Universidad de Valladolid (UVa) en el año 2019, siendo su nombre actual 'Online Judge'. Sin embargo, por razones de comodidad y mejor comprensión, mantendremos la abreviatura 'UVa' a lo largo del libro.

El Online Judge, uno de los jueces en línea más antiguos, cuenta con muchas herramientas desarrolladas por terceros para ayudar a sus usuarios, entre ellas uHunt, desarrollada por nosotros, (`https://uhunt.onlinejudge.org/`) y uDebug (`https://www.udebug.com/`).

Figura 1.2: I: Online Judge – D: Kattis.

Kattis (`https://open.kattis.com`, [34]) es el sistema de evaluación de las finales mundiales del ICPC más recientes. Cuenta con un juez en línea, público y abierto, que contiene problemas interesantes de las finales regionales y mundiales del ICPC, así como de otros concursos de buena calidad. En vez de clasificar a los usuarios por el número de problemas resueltos, como hace el Online Judge, Kattis utiliza un sistema 'dinámico' de calificación de la dificultad de los problemas. Esto supone que un programador competitivo muy bueno pueda escalar rápidamente en la clasificación, resolviendo intencionadamente problemas más complejos que los competidores que solo sean capaces de abordar problemas más sencillos y con una calificación menor. A fecha de 19 de julio de 2020, es necesario lograr $\geq$ 1792,7 puntos para situarse entre los 100 mejores usuarios. Steven se encuentra en la posición 9 (con 5742,7 puntos), de un total de $\approx$ 141 132 usuarios, es decir, también en el percentil 99,9.

En esta edición, utilizamos indistintamente UVa y Kattis como fuentes primarias de problemas.

Figura 1.3: I: USACO – D: ICPC Live Archive

La USA Computing Olympiad tiene una página web de entrenamiento muy potente [44], con concursos en línea que te ayudarán a mejorar tus habilidades de programación y resolución de problemas. Está más orientada a concursos de la IOI que del ICPC. Puedes ir directamente a su página web y empezar a entrenar.

La página 'hermana' del juez en línea de la UVa, es el ICPC Live Archive [30], contiene *casi todos* los problemas de las regionales y finales mundiales del ICPC, de los años 1990-2018. Puedes entrenar aquí, si quieres tener un buen rendimiento en ICPC futuros. Algunos de los problemas del Live Archive se pueden encontrar también en el Online Judge y, los más recientes, en Kattis.

Codeforces [4] y Topcoder [29] suelen organizar concursos de programación en línea, sin restricciones en cuanto a la edad de los participantes. Utilizan un sistema de clasificación (programadores rojos, naranjas, violetas, azules, cian, etc.) para recompensar con un nivel superior a los concursantes que resuelven problemas en las condiciones de presión del entorno de un concurso, frente a aquellos más diligentes que resuelven problemas más sencillos a lo largo de (mucho) más tiempo, pero con menos presión, y con ayuda de las explicaciones de solución de los problemas, que se hacen públicas una vez finalizado el concurso.

### 1.3.7 Consejo 7: trabaja en equipo (para ICPC)

Este último consejo es más fácil de plantear que de llevar a la práctica, pero aquí proponemos algunas ideas que podrían mejorar el rendimiento de tu equipo:

- Practicar la programación sobre el papel, ya sea con código o con pseudocódigo. Es muy útil cuando un compañero de equipo está utilizando el ordenador para que, al llegar tu turno, puedas programar lo más rápidamente posible.
- La estrategia de 'enviar e imprimir': si el código recibe un veredicto AC, basta con ignorar la copia impresa pero, si todavía no has llegado a ese punto, puedes depurar el código utilizando la versión en papel, mientras otro miembro del equipo está en el ordenador ocupado con otro problema. Cuidado, porque depurar código sin ayuda del ordenador es una tarea sumamente complicada. Podrías plantearte ser profesor ayudante en un curso de metodología *básica* de programación para desarrollar la capacidad de identificar los pequeños errores en los códigos de otros (y, al mismo tiempo, evitar cometerlos tú mismo).
- Si un compañero de equipo está programando (y tú no estás ocupado con otro problema), aprovecha el tiempo en preparar casos de prueba límite (que ojalá el programa de tu compañero pueda resolver). Cuando dos miembros del equipo coinciden en que un código es correcto, aumenta la posibilidad de evitar tiempo de penalización.
- Si eres consciente de que otro miembro del equipo es (significativamente) más apropiado para resolver un tipo de problema concreto que acabas de identificar, considera la posibilidad de pasárselo en vez de empeñarte en resolverlo tú.
- Practica la solución de problemas largos y complejos entre dos o, incluso, tres miembros del equipo, en preparación de la posibilidad de lograr un último acierto en los minutos finales del concurso.
- El factor X: mantén una buena relación de amistad con tus compañeros de equipo *fuera* de las sesiones de entrenamiento y los concursos.

## 1.4 Primeros pasos: los problemas fáciles

Nota: si tienes experiencia como participante en concursos de programación, puedes ignorar esta sección, que está dirigida a los recién llegados al mundo de la programación competitiva.

### 1.4.1 Anatomía de un problema

Un problema de un concurso de programación contiene *normalmente* los siguientes elementos:

- **`Historieta/enunciado del problema.`** La mayoría de los enunciados son atractivos. Sin embargo, los problemas más sencillos se suelen escribir de forma que *engañen* a los concursantes, aparentando ser difíciles, mediante la inclusión, por ejemplo, de 'información extra' para crear una distracción. Los concursantes deberían ser capaces de *filtrar* los detalles sin importancia y centrarse en los esenciales. Por ejemplo, todos los párrafos de

introducción, con la excepción de la última frase, del problema UVa 00579 - ClockHands cuentan la historia del reloj, y no tienen nada que ver con el problema propiamente dicho. En contraste, los problemas más difíciles suelen estar planteados de forma más aséptica, puesto que ya incluyen suficientes retos sin necesidad de adornos.

- **Descripción de la entrada y salida.** En esta sección, se proporcionan detalles del formato que tendrán los datos de entrada y del que deberían tener los de salida. Esta parte suele estar escrita en un estilo formal. Un buen problema debería determinar, claramente, cuáles son sus límites en la entrada, ya que podría resolverse con diferentes algoritmos, en función de dichos límites (ver la tabla 1.4).

- **Ejemplos de entrada y salida.** Los autores de los problemas suelen proporcionar casos de prueba triviales a los concursantes, como los del **ejercicio 1.1.1**. Los ejemplos de entrada y salida sirven para verificar que los concursantes han entendido el problema en lo más básico y para comprobar si el código es, al menos, capaz de procesar la entrada propuesta y proporcionar la salida correcta en el formato indicado. Nunca envíes código al juez si no es capaz de resolver el caso de ejemplo. En la sección 1.3.5, se trata el tema de las pruebas que se deben realizar al código antes de su envío.

- **Pistas o notas al pie** En algunos casos, los autores del problema pueden incluir pistas o notas al pie, para facilitar la comprensión del problema.

## 1.4.2 Rutinas típicas de entrada/salida

### Casos de prueba múltiples

En un problema de un concurso de programación, la correción del código viene normalmente determinada por la ejecución de dicho código contra *varios* casos de prueba. En vez de utilizar varios archivos de prueba individuales, algunos concursos de programación[30] utilizan *un* archivo de prueba con varios casos incluidos. En esta sección utilizaremos un problema muy sencillo, como ejemplo para casos de prueba múltiples: dados dos enteros pequeños ($\leq 100$) expresados en una línea, devolver su suma en una línea. Ilustramos, a continuación, tres posibles formatos de entrada/salida[31]:

1. El número de casos de prueba aparece en la primera línea de la entrada.

2. Los casos de prueba múltiples terminan con un valor especial (normalmente ceros), con independencia de que aparezcan más datos después.

3. Los casos de prueba múltiples terminan con la señal EOF (fin de archivo).

[30] El juez Kattis [34] prefiere la utilización de un archivo por caso de prueba.
[31] La lista no pretende ser exhaustiva.

| Código fuente C/C++ | Ejemplo de entrada | Ejemplo de salida |
|---|---|---|
| `int TC;`<br>`scanf("%d", &TC); // número de casos`<br>`while (TC--) { // repetir hasta llegar a 0`<br>`  int a, b; scanf("%d %d", &a, &b);`<br>`  printf("%d\n", a + b);  // calcular al vuelo`<br>`}` | 3<br>1 2<br>5 7<br>6 3<br>EOF | 3<br>12<br>9<br>EOF |
| `int a, b;`<br>`// parar cuando los enteros sean 0`<br>`while (scanf("%d %d", &a, &b), (a \|\| b))`<br>`  printf("%d\n", a + b);`<br>`  // no procesar esta última línea ->` | 1 2<br>5 7<br>6 3<br>0 0<br>1 1<br>EOF | 3<br>12<br>9<br>EOF |
| `int a, b;`<br>`// scanf devuelve el número de elementos`<br>`while (scanf("%d %d", &a, &b) == 2)`<br>`// o podemos comprobar el EOF, p.e.`<br>`// while (scanf("%d %d", &a, &b) != EOF)`<br>`  printf("%d\n", a + b);` | 1 2<br>5 7<br>6 3<br>EOF | 3<br>12<br>9<br>EOF |

### Números de caso y líneas en blanco

Algunos problemas, con casos de prueba múltiples, requieren que la salida de cada caso esté numerada de forma secuencial. Otros requieren, además, una línea en blanco *después* de cada caso de prueba. Vamos a modificar el problema anterior para incluir el número de caso en la salida (contando a partir de 1) con el siguiente formato: "`Case [NÚMERO]: [RESPUESTA]`" seguido de una línea en blanco por cada caso. Asumiendo que la entrada finaliza con la señal EOF, podemos utilizar el siguiente código:

| Código fuente C/C++ | Ejemplo de entrada | Ejemplo de salida |
|---|---|---|
| `int a, b, c = 1;`<br>`while (scanf("%d %d", &a, &b) != EOF)`<br>`  // ten en cuenta los dos '\n'`<br>`  printf("Case %d: %d\n\n", c++, a + b);` | 1 2<br>5 7<br>6 3<br>EOF | `Case 1: 3`<br><br>`Case 2: 12`<br><br>`Case 3: 9`<br><br>EOF |

Otros problemas nos pedirán que escribamos una línea en blanco solo *entre* casos de prueba. Si utilizamos la solución anterior, tendremos una línea extra al final de nuestra salida, lo que resultará en un veredicto de error de presentación (PE)[32]. Deberíamos utilizar el siguiente código:

[32]Algunos jueces, como Kattis, ignoran estas pequeñas, a la par que molestas, diferencias en el formato.

| Código fuente C/C++ | Ejemplo de entrada | Ejemplo de salida |
|---|---|---|
| `int a, b, c = 1;`<br>`while (scanf("%d %d", &a, &b) != EOF) {`<br>`  if (c > 1) printf("\n"); // caso 2 y sig.`<br>`  printf("Case %d: %d\n", c++, a + b);`<br>`}` | `1 2`<br>`5 7`<br>`6 3`<br>`EOF` | `Case 1: 3`<br><br>`Case 2: 12`<br><br>`Case 3: 9`<br>`EOF` |

### Número de entradas variable

Modifiquemos ligeramente el problema anterior. Ahora, para cada caso de prueba (cada línea de entrada) recibiremos un entero $k$ ($k \geq 1$), seguido de $k$ enteros. Nuestra tarea consistirá en mostrar la suma de esos $k$ enteros. Si damos por hecho que la entrada terminará con la señal EOF y que no debemos numerar los casos, nuestro código podría quedar así:

| Código fuente C/C++ | Ejemplo de entrada | Ejemplo de salida |
|---|---|---|
| `int k;`<br>`while (scanf("%d", &k) != EOF) {`<br>`  int ans = 0, v;`<br>`  while (k--) { scanf("%d", &v); ans += v; }`<br>`  printf("%d\n", ans);`<br>`}` | `1 1`<br>`2 3 4`<br>`3 8 1 1`<br>`4 7 2 9 3`<br>`5 1 1 1 1 1`<br>`EOF` | `1`<br>`7`<br>`10`<br>`21`<br>`5`<br>`EOF` |

La rutina de entrada puede ser *un poco más* problemática si no recibimos el entero $k$ al principio de cada línea/caso. Para realizar la misma tarea, asumiendo que $k \geq 1$ y que dos enteros en una misma línea estarán separados por, *exactamente*, un espacio, deberemos leer pares de un entero y otro carácter y detectar la señal de fin de línea (EOLN), por ejemplo:

| Código fuente C/C++ | Ejemplo de entrada | Ejemplo de salida |
|---|---|---|
| `while (1) { // bucle infinito`<br>`  int ans = 0, v;`<br>`  char dummy;`<br>`  while (scanf("%d%c", &v, &dummy) != EOF) {`<br>`  ans += v;`<br>`  if (dummy == '\n') break; // buscar EOLN`<br>`  }`<br>`  if (feof(stdin)) break; // buscar EOF`<br>`  printf("%d\n", ans);`<br>`}` | `1`<br>`3 4`<br>`8 1 1`<br>`7 2 9 3`<br>`1 1 1 1 1`<br>`EOF` | `1`<br>`7`<br>`10`<br>`21`<br>`5`<br>`EOF` |

Hemos escrito este mismo código de ejemplo en varios lenguajes de programación. Consúltalo en nuestro repositorio público de GitHub (`https://github.com/stevenhalim/cpbook-code`), especialmente si C/C++ no es tu lenguaje habitual.

| | |
|---|---|
| **C++** | `ch1/IO.cpp` |
| **Java** | `ch1/IO.java` |
| **Python** | `ch1/IO.py` |
| **OCaml** | `ch1/IO.ml` |

### 1.4.3 Empieza el viaje

No hay mejor forma de empezar el viaje en el mundo de la programación competitiva que resolviendo problemas. Para ayudarte a elegir con cuáles empezar, del total de ≈ 4965 problemas ofrecidos en el Online Judge [45] y de los ≈ 2746 disponibles en Kattis [34], hemos preparado una lista con algunos de los problemas *ad hoc* más sencillos. Las primeras cinco categorías se encuentran entre las más sencillas que cabría encontrar en un concurso de programación, y son adecuadas incluso para estudiantes noveles de ciencias de la computación, que acaban de empezar a conocer los rudimentos básicos de metodologías de programación, pero tienen algunas nociones de matemáticas (como manipulaciones algebraicas sencillas o aritmética modular elemental) y lógica (operaciones Y, O, NO). Si estás empezando en el mundo de la programación competitiva, te recomendamos que trates de resolver algunos problemas de esta categoría después de haber finalizado la lectura de la sección 1.4.2.

Como cada categoría contiene muchos problemas que podrían ser apabullantes para quienes están empezando, hemos *resaltado* un problema de nivel básico (ya sea de Kattis o de UVa) junto a, normalmente, tres **obligatorios *** de cada uno de los dos jueces en cada categoría. Consideramos que estos son problemas interesantes, o de mejor calidad, y no implican formatos complicados en la entrada y la salida. El resto de problemas mostrados aparecen como contenido adicional. La lista completa de consejos para resolver los problemas resaltados *y los adicionales* se encuentra disponible en `https://cpbook.net`.

- **Solo E/S y/o secuencias**
  La mayoría de estos problemas se pueden resolver en un par de líneas de código.
- **Solo repetición**
  Todos los problemas de esta categoría se resuelven con sentencias de repetición (bucle `for`, bucle con rango `for`, bucle `while` o bucle `do while`).
- **Solo selección**
  Todos los problemas de esta categoría se pueden resolver con sentencias de selección (`if/else if/else` o `switch/case`).
- **Solo repetición y selección**
  Son problemas básicamente de selección pero con casos de prueba múltiple, lo que implica un bucle exterior (una sentencia de repetición).
- **Lineales**
  Tenemos E/S, secuencias, selección y repetición juntos. Los problemas de esta categoría se pueden resolver sin utilizar *arrays*.
- **Función**
  Los problemas de esta categoría contienen partes que se pueden abstraer en funciones (*definidas por el usuario*), incluyendo funciones recursivas, que pueden necesitar *más de una llamada*.
- **Manipulación de *arrays* unidimensionales, muy fáciles**
  Los problemas de esta categoría son más fáciles de resolver si utilizamos una estructura de datos de *array* unidimensional.
- **Fáciles, menos fáciles y medios**
  Los problemas de las tres siguientes categorías son fáciles, menos fáciles y medios, en los

que no encontraremos un formato de E/S complicado[33]. Pero, de aquí en adelante, los problemas que mencionaremos incluirán una combinación de las técnicas metodológicas básicas que acabamos de mencionar.

### 1.4.4 Nuestro primer veredicto de aceptado (AC)

En esta subsección, te guiaremos en el camino de lograr tu primer veredicto de aceptado (AC), utilizando un sencillo problema. Puedes ignorar esta parte si ya has obtenido tus primeros éxitos en este sentido. Utilizaremos el problema Kattis - moscowdream para ilustrar, paso a paso, el proceso de solución.

#### Primer intento

Una vez eliminada la información irrelevante, el problema Kattis - moscowdream se puede resumir de la siguiente manera: "dados 4 enteros $a$, $b$, $c$ y $n$ ($0 \leq a, b, c \leq 10$, $1 \leq n \leq 20$), escribir '`YES`' en la salida si $a \geq 1$, $b \geq 1$, $c \geq 1$ y $a + b + c \geq n$, o '`NO`' en caso contrario".

La mayoría de los concursantes inexpertos en programación competitiva escribirán un código en C/C++ similar al siguiente, y lo enviarán al juez.

```
#include <stdio.h>
int main() {
  int a, b, c, n; scanf("%d %d %d %d", &a, &b, &c, &n);
  if ((a >= 1) && (b >= 1) && (c >= 1) && (a+b+c >= n))
    printf("YES\n");
  else
    printf("NO\n");
  return 0;
}
```

#### Segundo intento

Por desgracia, el código anterior provocará un veredicto de respuesta incorrecta (WA). Si hubieses participado en un concurso, habrías observado que la mayoría de equipos habrían resuelto este sencillo problema pero, muchos de ellos, habrían necesitado un *segundo* envío para lograrlo. Esto implica que el autor del problema ha utilizado, muy probablemente, algún caso límite para atrapar a los concursantes que hayan bajado la guardia. Vamos a ejecutar nuestro programa con algunos casos de prueba aleatorios, diferentes del proporcionado en el ejemplo. Resulta que el caso `1 1 1 1` devuelve '`YES`'. En este punto deberíamos ser conscientes de que la respuesta correcta debería ser '`NO`' ya que, cuando $n = 1$, es imposible tener, al menos, 1 problema fácil, 1 problema medio y 1 problema difícil. Hemos ignorado el caso en el que $1 \leq n < 3$. Debemos observar que el autor del problema ha escrito, astutamente, $1 \leq n \leq 20$ como límite de entrada de la variable $n$.

[33]Una vez hayamos visto técnicas básicas de procesamiento de cadenas, en la sección 1.5, mostraremos muchos más problemas *ad hoc* en la sección 1.6

Denominamos a esta situación un problema de "análisis de casos", es decir, el problema cuenta con casos que pueden (o deben) ser tratados de forma independiente. Para este sencillo problema es muy posible que depurar el fallo no tenga mucha complicación. Sin embargo, en problemas más difíciles, la identificación de casos límite podría no ser tan evidente.

```
#include <bits/stdc++.h>                          // útil en concursos
using namespace std;                              // también útil
int main() {
  int a, b, c, n; scanf("%d %d %d %d", &a, &b, &c, &n);    // la corrección
  printf(((a >= 1) && (b >= 1) && (c >= 1) && (a+b+c >= n) && (n >= 3)) ?
         "YES\n" : "NO\n");                       // usar operador ternario
  return 0;                                       // para abreviar el código
}
```

| | |
|---|---|
| **C++** | `ch1/moscowdream.cpp` |
| **Java** | `ch1/moscowdream.java` |
| **Python** | `ch1/moscowdream.py` |
| **OCaml** | `ch1/moscowdream.ml` |

## Ejercicios de programación

**Ejercicios de programación para empezar**[34]**:**

**Solo E/S y secuencias**

1. Nivel básico: *Kattis - hello ** — basta escribir "Hello World!"
2. **UVa 10071 - Back to High School ... *** — muy sencillo: escribir $2 \times v \times t$
3. **UVa 11614 - Etruscan Warriors ... *** — raíz de una ecuación cuadrática
4. **UVa 13025 - Back to the Past *** — de regalo, escribir una línea de respuesta
5. *Kattis - carrots ** — basta escribir P
6. *Kattis - r2 ** — basta escribir $2 \times S - R1$
7. *Kattis - thelastproblem ** — $S$ puede tener espacios

   Adicionales UVa: *11805*, *12478*.
   Adicionales Kattis: *faktor*, *planina*, *romans*.

**Solo repetición**

1. Nivel básico: *Kattis - timeloop ** — escribir 'num Abracadabra' N veces
2. **UVa 01124 - Celebrity Jeopardy *** — LA 2681 - SouthEasternEurope06, reproducir la entrada
3. **UVa 11044 - Searching for Nessy *** — existe código/fórmula de una línea
4. **UVa 11547 - Automatic Answer *** — existe solución de una línea en $O(1)$
5. *Kattis - different ** — usar la función `abs` por cada caso de prueba
6. *Kattis - qaly ** — bucle trivial
7. *Kattis - tarifa ** — una pasada, no es necesario *array*

   Adicionales UVa: *10055*.

**Solo selección**

1. Nivel básico: *Kattis - moscowdream ** `if-else`, 2 casos, comprobar $n \geq 3$
2. *Kattis - isithalloween ** `if-else`, 2 casos
3. *Kattis - judgingmoose ** `if-else if-else`, 3 casos
4. *Kattis - onechicken ** `if-else if-else`, 4 casos (pieza contra piezas)
5. *Kattis - provincesandgold ** `if-else if-else`, 6 casos
6. *Kattis - quadrant ** `if-else if-else`, 4 casos
7. *Kattis - temperature ** `if-else if-else`, 3 casos, deducir fórmula

**Casos de prueba múltiples y selección**

1. Nivel básico: *Kattis - oddities ** 2 casos
2. **UVa 11172 - Relational Operators *** muy fácil, una línea
3. **UVa 12250 - Language Detection *** LA 4995 - KualaLumpur10, `if-else`
4. **UVa 12372 - Packing for Holiday *** comprobar si $L, W, H \leq 20$
5. *Kattis - eligibility ** 3 casos
6. *Kattis - helpaphd ** 2 casos
7. *Kattis - leftbeehind ** 4 casos

Adicionales UVa: *00621*, *11723*, *11727*, *12289*, *12468*, *12577*, *12646*, *12917*.
Adicionales Kattis: *nastyhacks*, *numberfun*.

**Lineales (se resuelven en menos de 7 minutos[35])**

1. Nivel básico: *Kattis - statistics ** una pasada, no es necesario *array*
2. **UVa 11764 - Jumping Mario *** una pasada lineal para contar los saltos alto+bajo
3. **UVa 11799 - Horror Dash *** una pasada lineal, hallar el valor máximo
4. **UVa 12279 - Emoogle Balance *** pasada lineal sencilla
5. *Kattis - fizzbuzz ** en realidad es sobre propiedades de divisibilidad sencillas
6. *Kattis - licensetolaunch ** pasada lineal sencilla
7. *Kattis - oddgnome ** pasada lineal

Adicionales UVa: *00272*, *10300*, *11364*, *11498*, *12403*, *13012*, *13034*, *13130*.
Adicionales Kattis: *babybites*, *cold*, *earlywinter*, *jobexpenses*, *speedlimit*, *stararrangements*, *thanos*, *zanzibar*.

**Función**

1. Nivel básico: *Kattis - mia ** solo una comprobación `if-else`
2. **UVa 10424 - Love Calculator *** hacer lo que se pide
3. **UVa 11078 - Open Credit System *** una pasada lineal, función `max`
4. **UVa 11332 - Summing Digits *** recursivo sencillo
5. *Kattis - artichoke ** LA 7150 - WorldFinals Marrakech15, pasada lineal, también disponible en UVa 01709 - Amalgamated Artichokes
6. *Kattis - digits ** simulación directa, también disponible en UVa 11687 - Digits
7. *Kattis - filip ** función de creación de una 'cadena inversa', después comprobación `if-else`

Adicionales Kattis: *abc*, *combinationlock*, *treasurehunt*.

**Manipulación de *arrays* unidimensionales, muy fáciles**

1. Nivel básico: *Kattis - lostlineup ** — manipulación de *array* sencilla
2. **UVa 01585 - Score *** — LA 3354 - Seoul05, algoritmo de una pasada muy sencillo
3. **UVa 11679 - Sub-prime *** — simular, ver si todos los bancos tienen una reserva $\geq 0$
4. **UVa 12015 - Google is Feeling Lucky *** — recorrer la lista dos veces
5. *Kattis - acm ** — simulación sencilla, una pasada, guardar número de WA por problema
6. *Kattis - cetiri ** — ordenar 3 números ayuda, 3 casos
7. *Kattis - lineup ** — ordenación creciente/decreciente y comparar

Adicionales UVa: *11942*.
Adicionales Kattis: *basketballoneonone*, *hothike*.

**Fáciles**

1. Nivel básico: *Kattis - hissingmicrophone ** — bucle sencillo
2. **UVa 12503 - Robot Instructions *** — simulación sencilla
3. **UVa 12658 - Character Recognition? *** — comprobación de reconocimiento de caracteres
4. **UVa 12696 - Cabin Baggage *** — LA 6608 - Phuket13, problema sencillo
5. *Kattis - batterup ** — un bucle sencillo
6. *Kattis - hangingout ** — bucle sencillo
7. *Kattis - pokerhand ** — conteo de frecuencia, escribir máximo

Adicionales UVa: *01641*, *10963*, *12554*, *12750*, *12798*.
Adicionales Kattis: *armystrengtheasy*, *armystrengthhard*, *brokenswords*, *drinkingsong*, *mosquito*, *ptice*, *sevenwonders*, *volim*, *yinyangstones*.
Otros: IOI 2010 - Cluedo (3 punteros), IOI 2010 - Memory (2 pasadas lineales).

**Menos fáciles**

1. Nivel básico: *Kattis - bubbletea ** — simulación sencilla
2. **UVa 11559 - Event Planning *** — una pasada lineal
3. **UVa 11683 - Laser Sculpture *** — una pasada lineal es suficiente
4. **UVa 11786 - Global Raining ... *** — debes observar el patrón
5. *Kattis - bossbattle ** — pregunta con trampa
6. *Kattis - peasoup ** — una pasada lineal
7. *Kattis - vote ** — sigue las instrucciones

Adicionales UVa: *10114*, *10141*, *10324*, *11586*, *11661*, *12614*, *13007*.
Adicionales Kattis: *boundingrobots*, *climbingstairs*, *deathtaxes*, *driversdilemma*, *eventplanning*, *exactlyelectrical*, *missingnumbers*, *prerequisites*, *sok*.

**Medios**

1. Nivel básico: *Kattis - basicprogramming1 ** — un interesante problema de sumas para un examen de un curso de metodología básica de programación
2. **UVa 11507 - Bender B. Rodriguez ... *** — simulación, `if-else`
3. **UVa 12157 - Tariff Plan *** — LA 4405 - KualaLumpur08, calcular y comparar los dos planes
4. **UVa 12643 - Tennis Rounds *** — tiene casos de prueba complicados
5. *Kattis - battlesimulation ** — una pasada, comprobación especial en $3! = 6$ posibles combinaciones de 3 movimientos combinados

6. *Kattis - bitsequalizer ** análisis de patrones, también disponible en UVa 12545 - Bits Equalizer
7. *Kattis - fastfood ** debido a las restricciones se hace en una pasada

Adicionales UVa: *00119*, *00573*, *00661*, *01237*, *11956*.

Adicionales Kattis: *anotherbrick*, *beekeeper*, *bottledup*, *carousel*, *climbingworm*, *codecleanups*, *cowcrane*, *howl*, *shatteredcake*.

Otros: IOI 2009 - Garage (simulación), IOI 2009 - POI (ordenación).

Después de haber resuelto un buen número de problemas de programación, comenzarás a identificar un patrón en tus

capítulo o en el código fuente disponible en `https://github.com/stevenhalim/cpbook-code`). Si ninguna de nuestras opciones te sorprende (o, incluso, las has mejorado), significa que estás en buena forma para lidiar con varios problemas de procesamiento de cadenas, por lo que podrás pasar a las siguientes secciones. En caso contrario, dedica algo de tiempo a estudiar nuestras versiones.

1. Dado un archivo de texto que contiene únicamente caracteres alfabéticos `[A-Za-z]`, dígitos `[0-9]`, el espacio y el punto ('.'), escribe un programa que lea el mencionado archivo, línea a línea, hasta que encuentre una línea que *comience* con siete puntos ("`.......`"). Concatena (combina) cada línea en un cadena más larga `T`. Cuando se combinen dos líneas, coloca un espacio entre ellas, de forma que la última palabra de la primera línea quede separada de la primera palabra de la segunda. En la entrada puede haber hasta 30 caracteres por línea y no más de 10 líneas. No hay un espacio al final de cada línea y todas terminan con el carácter de salto de línea. Mostramos la entrada de ejemplo después de la pregunta 1.d y antes de la tarea 2.

   *a*) ¿Sabes cómo almacenar una cadena en tu lenguaje de programación favorito?

   *b*) ¿Cómo leer un texto de entrada línea a línea?

   *c*) ¿Cómo concatenar (combinar) dos cadenas en una más larga?

   *d*) ¿Cómo comprobar si una línea comienza con '`.......`', para no leer más la entrada?

```
I love CS3233 Competitive
Programming. i also love
AlGoRiThM
.......you must stop after reading this line as it starts with 7 dots
after the first input block, there will be one looooooooooooong line...
```

2. Supongamos que tenemos una cadena larga `T`. Queremos comprobar si otra cadena `P` se encuentra dentro de `T`, notificar todos los índices donde `P` aparece en `T`, o indicar `-1` si `P` no se encuentra en `T`. Por ejemplo, si `T` = "`I love CS3233 Competitive Programming. i also love AlGoRiThM`" y `P` = "`I`", la salida será, únicamente, {0} (con un índice que comienza en 0) porque la mayúscula '`I`' y la minúscula '`i`' se consideran diferentes, por lo que el carácter '`i`' de la posición {39} no formará parte de la salida. Si `P` = "`love`", la salida será {2, 46}. Si `P` = "`book`", el resultado mostrará {-1}.

   *a*) ¿Cómo encontrar la primera aparición de una subcadena en una cadena (si existe)? ¿Necesitamos implementar un algoritmo de coincidencia de cadenas (como el de Knuth–Morris–Pratt, tratado en el Volumen II, etc.) o será suficiente con utilizar funciones de una biblioteca?

   *b*) ¿Cómo encontrar el resto de apariciones de una subcadena en un cadena (si existen)?

3. Supongamos ahora que queremos hacer un análisis sencillo del contenido de `T`, y convertir cada uno de sus caracteres en minúsculas. ¿El análisis implicará contar el número de dígitos, vocales `[aeiouAEIOU]` y consonantes (cualquier letra que no sea una vocal) que hay en `T`? ¿Es posible realizar todas las operaciones en $O(n)$, donde $n$ es la longitud de la cadena `T`?

4. A continuación, queremos dividir la cadena larga `T` en *trozos* (subcadenas) y almacenarlos en un *array* de cadenas llamado `tokens`. Para esta tarea, los *delimitadores* de los bloques serán los espacios en blanco y los signos de puntuación (por lo tanto, dividiremos las frases en palabras). Por ejemplo, si queremos *trocear* la cadena `T` (en minúsculas), obtendremos los siguientes `tokens` = {"`i`", "`love`", "`cs3233`", "`competitive`", '`programming`", "`i`", "also", "`love`", "`algorithm`"}. Después, ordenaremos lexicográficamente este *array* de cadenas[38] y buscaremos la cadena de menor valor lexicográfico. Es decir, habremos ordenado `tokens`: {"`algorithm`", "`also`", "`competitive`", "`cs3233`", "`i`", "`i`", "`love`", "`love`", "`programming`"}. Por lo tanto, la cadena de menor valor lexicográfico de este ejemplo será "`algorithm`".

   *a*) ¿Cómo troceamos una cadena?

   *b*) ¿Cómo almacenamos los trozos (las cadenas más cortas) en un *array* de cadenas?

   *c*) ¿Cómo ordenamos lexicográficamente un *array* de cadenas?

5. Ahora, identifiquemos la palabra que aparece más veces en `T`. Para resolver esta cuestión, necesitamos contar la frecuencia con la que aparece cada palabra. En el caso de `T`, la salida será "`i`" o "`love`", ya que ambas aparecen dos veces. ¿Qué estructura de datos deberíamos utilizar para esta pequeña tarea?

6. El texto dado tiene una línea más, después de la que comienza con "`.....`", pero la longitud de esta última línea no está limitada. La tarea consistirá en contar cuántos caracteres hay en ella. ¿Cómo podemos leer una cadena si no conocemos su longitud por adelantado?

| | |
|---|---|
| **Tarea** | `ch1/basic_string.html` |
| **C++** | `ch1/basic_string.cpp` |
| **Java** | `ch1/basic_string.java` |
| **Python** | `ch1/basic_string.py` |
| **OCaml** | `ch1/basic_string.ml` |

## 1.6 Los problemas *ad hoc*

Finalizaremos este capítulo introductorio tratando el primer tipo de problema en los ICPC y en las IOI: los problemas *ad hoc*. Según la USACO [44], los problemas *ad hoc* son aquellos que 'no pueden clasificarse en ninguna otra categoría', ya que cada enunciado y su correspondiente solución son 'únicos'. Muchos problemas *ad hoc* son fáciles (como los que aparecen en la sección 1.4), pero esto no se aplica necesariamente a todos ellos.

Los problemas *ad hoc* aparecen frecuentemente en los concursos de programación. En el ICPC, ≈ 1-2 de cada ≈ 10-13 son *ad hoc*. Si el problema es fácil, será normalmente el primero que resuelvan los equipos participantes en el concurso. Sin embargo, hay casos en los que las soluciones a los problemas *ad hoc* son demasiado complicadas de implementar, lo que provoca que, por estrategia, algunos equipos prefieran abordarlos al final del concurso. En una competición regional del ICPC con unos 60 equipos, tu equipo se clasificará en la mitad baja si *solo* conseguís resolver los problemas *ad hoc* (fáciles).

[38] Que, básicamente, consiste en ordenar las palabras como aparecerían en un diccionario.

En las IOI más recientes[39], han aparecido cada vez más tareas *ad hoc* que necesitan creatividad para su solución [20]. Cuantos más problemas *ad hoc* resuelvas, más ampliarás tus conocimientos y posibilidades de resolver otros.

Hemos hecho una lista de categorías de **muchos** problemas *ad hoc* que hemos resuelto tanto en el Online Judge como en Kattis [45, 34]. Consideramos que es posible resolver la mayoría de estos problemas *sin* utilizar estructuras de datos avanzadas o los algoritmos que estudiaremos en los siguientes capítulos, es decir, debería bastarnos con leer cuidadosamente los requisitos del enunciado y programar la solución, que habitualmente será corta. Muchos de estos problemas *ad hoc* son 'sencillos', pero algunos pueden 'tener truco'. Algunos problemas *ad hoc* podrían necesitar de las habilidades básicas de procesamiento de cadenas que hemos visto en la sección 1.5. Intenta resolver *alguno de cada categoría* antes de pasar al próximo capítulo.

Hay un pequeño conjunto de problemas que, aunque aparece como parte del capítulo 1, puede requerir conocimientos de los siguientes capítulos como, por ejemplo, las estructuras de datos lineales (*arrays*) de la sección 2.2, el *backtracking* de la sección 3.2, etc. Puedes revisitar los problemas *ad hoc* más difíciles una vez que entiendas los conceptos necesarios.

Las categorías:

- **Juegos (naipes)**:
  Hay muchos problemas *ad hoc* relacionados con juegos populares, bastantes de ellos de naipes. Normalmente, tendrás que procesar las cadenas de entrada (ver la sección 1.5 si no estás familiarizado con esta técnica) como naipes que tienen tanto palos (D/Diamantes/♢, T/Tréboles/♣, C/Corazones/♡ y P/Picas/♠) como números (normalmente: $2 < 3 < \cdots < 9 <$ D/Diez < J/Jota < Q/Reina < K/Rey < A/Ás[40]). Puede ser una buena idea convertir estas cadenas a índices de enteros, para evitar problemas. Por ejemplo, un mapa posible sería convertir D2 → 0, D3 → 1, …, DA → 12, T2 → 13, T3 → 14, …, PA → 51. A partir de ahí, se puede trabajar solo con los índices enteros.

- **Juegos (ajedrez)**:
  El ajedrez es otro popular juego que, en ocasiones, aparece en los problemas de los concursos de programación. Algunos de estos problemas son *ad hoc* y aparecen en esta sección. Otros son de combinatoria, con tareas como contar cuántas formas hay de colocar 8 reinas en un tablero de ajedrez de $8 \times 8$. Esos los trataremos en el capítulo 3 y otros.

- **Juegos (otros)**, fáciles y difíciles (o más tediosos):
  Aparte de los juegos de naipes y de ajedrez, muchos otros juegos populares han encontrado un lugar en los concursos de programación: tres en raya, piedra-papel-tijera, serpientes y escaleras, bingo, bolos, etc. Conocer las reglas de estos juegos puede resultar útil[41] aunque, la mayoría de las veces, la información necesaria aparecerá en el enunciado del problema, para evitar perjudicar a los concursantes que no estén familiarizados con ellos.

- Problemas interesantes **de la vida real**, fáciles, medios y difíciles (o más tediosos):
  Esta es una de las categorías más relevantes de entre las que encontramos tanto en UVa como en Kattis. Creemos que los problemas de la vida real, como estos, son interesantes para cualquier recién llegado al mundo de la informática. El hecho de programar para

[39] La IOI utiliza ahora un sistema de subtareas, donde la primera de ellas, en cada tarea, suele ser la versión más sencilla. Si eres concursante de la IOI, lo más probable es que no logres ninguna medalla si solo resuelves las primeras subtareas de cada una de las tareas.

[40] En ocasiones A/Ás < 2.

[41] Y, otras veces, una desventaja si las reglas planteadas son *diferentes* a las originales.

resolver problemas de la vida real, puede ser una motivación adicional. Quién sabe, quizá puedas aprender algo nuevo (e interesante) al leer el enunciado del problema.

- Problemas *ad hoc* relacionados con el **tiempo**, fáciles y difíciles:
  Estos problemas utilizan conceptos temporales como fechas, horas y calendarios. También son problemas de la vida real. Como ya hemos mencionado, estos problemas pueden resultar un poco más interesantes de resolver. Algunos de ellos son mucho más fáciles de abordar si dominamos[42] el módulo `datetime` de Python o la clase `GregorianCalendar` de Java, ya que cuentan con muchas funciones para tratar con el tiempo. Por ejemplo, con `datetime` de Python podemos `+` (sumar una cierta cantidad de tiempo a una fecha), `-` (hallar la diferencia entre dos fechas), darle a las fechas el formato deseado, etc. En la clase de Java `GregorianCalendar`, podemos utilizar métodos como `add`, `get` (un componente de una fecha), `compareTo` (con otra fecha), etc.

- **Números romanos**:
  Los números romanos forman un sistema numérico utilizado en la antigua Roma. En realidad, es un sistema numérico decimal, pero utiliza ciertas letras del alfabeto, en vez de los dígitos `[0..9]` (se describen a continuación), no es posicional y no tiene un símbolo para el 0. En el sistema de números romanos hay 7 letras básicas, que acompañamos de sus valores decimales: I=1, V=5, X=10, L=50, C=100, D=500 y M=1000. También utilizan los siguientes pares de letras: IV=4, IX=9, XL=40, XC=90, CD=400, CM=900. Los problemas de programación relativos a números romanos suelen tratar de la conversión hacia y desde números arábigos (el sistema decimal que utilizamos habitualmente). Son problemas que aparecen muy raramente en concursos de programación, y el tipo de conversión se detecta al instante al leer el enunciado del problema. Si tienes interés en ver nuestra solución, puedes analizar el siguiente código:

| | |
|---|---|
| **C++** | `ch1/UVa11616.cpp` |
| **Java** | `ch1/UVa11616.java` |
| **Python** | `ch1/UVa11616.py` |

- **Cifrar/codificar/decodificar/descifrar**:
  A todo el mundo le gusta que sus comunicaciones digitales privadas sean seguras. Es decir, que sus mensajes (cadenas) solo puedan ser leídos por los destinatarios deseados. Se han inventado muchos sistemas de cifrado con este propósito y una buena parte (de los más sencillos, que suelen implicar *arrays* y/o bucles) acaban convirtiéndose en problemas *ad hoc* en concursos de programación, con sus diferentes reglas de codificado/decodificado. Se puede encontrar un buen número en el Online Judge [45] y en Kattis [34]. Así que vamos a dividir esta categoría en tres: fáciles, medios y difíciles (que hemos incorporado al Volumen II). Conviene intentar resolver algunos de ellos, especialmente los que clasificamos como **obligatorio ***. Es conveniente tener conocimientos sobre *seguridad informática y criptografía* para resolverlos.

- **Procesamiento de la entrada (iterativo)**:
  Este conjunto de problemas no está dirigido a los concursantes de la IOI, pues el temario actual obliga a que la entradas de las tareas tengan un formato lo más sencillo posible. Sin embargo, en el ICPC no existen tales restricciones. Los problemas de procesamiento

[42] C++ cuenta con `<ctime>`, pero tiene muchas menos funcionalidades que sus equivalentes en Python/Java.

pueden ir desde los más sencillos, que se pueden resolver con un procesador iterativo, hasta otros más complejos que requieren de procesadores descendentes recursivos, expresiones regulares de C++, clases de cadenas y patrones de Java y clases de expresiones regulares de Python y OCaml (dejamos los más complejos para el Volumen II).

- **Formato de salida**:
  Este es otro conjunto de problemas que tampoco es necesario para los concursantes de la IOI. En esta ocasión, el problema lo encontramos en la salida. En un conjunto de problemas del ICPC, este tipo de planteamientos se utilizan como 'calentamiento' o problemas para 'perder el tiempo'. Practica tu habilidad para resolver estos problemas *lo más rápidamente posible*, ya que el tiempo de penalización puede suponer una gran diferencia en el resultado final (veremos los más complejos en el Volumen II).

- Problemas para 'perder el tiempo':
  Estos problemas *ad hoc* están escritos, expresamente, para hacer que la solución sea larga y aburrida. Si aparecen en un concurso de programación, determinarán qué equipo cuenta con el programador más *eficiente*, alguien que es capaz de implementar soluciones complicadas y precisas bajo la presión del tiempo. Los entrenadores deberían incluir este tipo de problemas en sus sesiones de preparación.

- Problemas *ad hoc* en **otros capítulos**:
  Hay muchos problemas *ad hoc* que hemos desplazado a otros capítulos, ya que su solución requiere conocimientos que están por encima de las capacidades de programación básicas, pero podría ser una buena idea echarles un vistazo al terminar este capítulo 1.

  - Problemas *ad hoc* que requieren del uso de estructuras de datos lineales básicas (sobre todo *arrays*), en la sección 2.2.
  - Problemas *ad hoc* que requieren cálculos matemáticos, en el Volumen II.
  - Problemas *ad hoc* con procesamiento de cadenas *más complejo*, en el Volumen II.
  - Problemas *ad hoc* que requieren geometría básica, en el Volumen II.
  - Problemas *ad hoc* hoy día en desuso, como las Torres de Hanoi, en el capítulo 9.

## Ejercicios de programación

**Ejercicios de programación relacionados con problemas *ad hoc*:**

**Juegos (naipes)**

1. Nivel básico: **UVa 10646 - What is the Card?** * — barajar naipes según ciertas reglas y obtener uno determinado
2. **UVa 10388 - Snap *** — simulación de naipes, utiliza números aleatorios para determinar movimientos, necesita una estructura de datos para identificar los naipes boca arriba y boca abajo
3. **UVa 11678 - Card's Exchange *** — es solo un problema de manipulación de *arrays*
4. **UVa 12247 - Jollo *** — interesante juego de naipes, sencillo, pero que requiere un buen análisis lógico para lograr resolver todos los casos
5. *Kattis - bela* * — problema de puntuación de naipes sencillo
6. *Kattis - shuffling* * — simular la operación de mezclar naipes

7. *Kattis - memorymatch ** — juego de simulación interesante, muchos casos límite

   Adicionales UVa: *00162*, *00462*, *00555*, *10205*, *10315*, *11225*, *12366*, *12952*.
   Adicionales Kattis: *karte*.

**Juegos (ajedrez)**

1. Nivel básico: **UVa 00278 - Chess** * — es necesario un conocimiento básico del ajedrez, deducir la fórmula cerrada
2. **UVa 00255 - Correct Move *** — comprobar la validez de los movimientos de ajedrez
3. **UVa 00696 - How Many Knights *** — *ad hoc*, ajedrez
4. **UVa 10284 - Chessboard in FEN *** — FEN = Notación de Forsyth–Edwards, una notación estándar para describir posiciones en el tablero de ajedrez
5. *Kattis - chess ** — movimientos del alfil, imposibles o de 0, 1 o 2 maneras, utilizar fuerza bruta
6. *Kattis - empleh ** — el problema inverso a *Kattis - helpme **
7. *Kattis - helpme ** — convertir el tablero dado en notación de ajedrez

   Adicionales UVa: *10196*, *10849*, *11494*.
   Adicionales Kattis: *bijele*.
   Otros: ver también el problema de las $N$-reinas (sección 3.2.2 y en el Volumen II) y el problema de movimientos del caballo (sección 4.4.2) para otros ejercicios relacionados.

**Juegos (otros), fáciles**

1. Nivel básico: **UVa 10189 - Minesweeper** * — simular el juego clásico del buscaminas, similar a UVa 10279
2. **UVa 00489 - Hangman Judge *** — hacer lo que se pide
3. **UVa 00947 - Master Mind Helper *** — similar a UVa 00340
4. **UVa 11459 - Snakes and Ladders *** — simular, similar a UVa 00647
5. *Kattis - connectthedots ** — juego infantil clásico, formato de salida
6. *Kattis - gamerank ** — simular el proceso de actualización de la clasificación
7. *Kattis - guessinggame ** — utilizar un *array* de etiquetas unidimensional, también disponible en UVa 10530 - Guessing Game

   Adicionales UVa: *00340*, *10279*, *10409*, *12239*.
   Adicionales Kattis: *trik*.

**Juegos (otros), difíciles (más tediosos)**

1. Nivel básico: *Kattis - rockpaperscissors ** — contar victorias y derrotas, mostrar la media de victorias, también disponible en UVa 10903 - Rock-Paper-Scissors ...
2. **UVa 00584 - Bowling *** — simulación, juegos, comprensión lectora
3. **UVa 10813 - Traditional BINGO *** — seguir el enunciado del problema
4. **UVa 11013 - Get Straight *** — comprobar las permutaciones de 5 naipes para determinar la mejor secuencia, obtener el sexto naipe por fuerza bruta y sustituir uno de tus naipes con él
5. *Kattis - battleship ** — simulación, comprensión lectora, muchos casos límite
6. *Kattis - tictactoe2 ** — comprobar la validez del juego de las tres en raya, complicado, también disponible en UVa 10363 - Tic Tac Toe
7. *Kattis - turtlemaster ** — interesante juego de mesa para enseñar programación a niños, simulación

   Adicionales UVa: *00114*, *00141*, *00220*, *00227*, *00232*, *00339*, *00379*, *00647*.
   Adicionales Kattis: *rockscissorspaper*.

**Tiempo, fáciles**

**Cifrar/codificar/decodificar/descifrar, fáciles**

1. Nivel básico: **UVa 13145 - Wuymul Wixcha** * — desplazar los valores del alfabeto +6 caracteres para leer el enunciado del problema, problema sencillo de cifrado César
2. **UVa 10851 - 2D Hieroglyphs ... *** — ignorar límite, tratar '\/' como 1/0
3. **UVa 11278 - One-Handed Typist *** — vincular teclas QWERTY a DVORAK
4. **UVa 12896 - Mobile SMS *** — cifrado sencillo, utilizar mapeado
5. *Kattis - conundrum ** — cifrado sencillo
6. *Kattis - encodedmessage ** — cifrado sencillo en rejilla bidimensional
7. *Kattis - t9spelling ** — similar a (el inverso de) UVa 12896

Adicionales UVa: *00444*, *00641*, *00795*, *00865*, *01339*, *10019*, *10222*, *10878*, *10896*, *10921*, *11220*, *11541*, *11946*, *13107*.
Adicionales Kattis: *drmmessages*, *drunkvigenere*, *kemija08*, *keytocrypto*, *reverserot*, *runlengthencodingrun*.

**Cifrar/codificar/decodificar/descifrar, medios**

1. Nivel básico: *Kattis - secretmessage ** — hacer lo que se pide, usar rejilla bidimensional
2. **UVa 00245 - Uncompress *** — LA 5184 - WorldFinals Nashville95
3. **UVa 00492 - Pig Latin *** — *ad hoc*, similar a UVa 00483
4. **UVa 11787 - Numeral Hieroglyphs *** — seguir el enunciado
5. *Kattis - anewalphabet ** — cifrado sencillo, 26 caracteres
6. *Kattis - tajna ** — cifrado sencillo con rejilla bidimensional
7. *Kattis - piglatin ** — sencillo, comprobar las vocales que incluyen 'y' y procesarlo

Adicionales UVa: *00483*, *00632*, *00739*, *00740*, *11716*.
Adicionales Kattis: *falsesecurity*, *permcode*.

**Procesamiento de la entrada (iterativo)**

1. Nivel básico: **UVa 11878 - Homework Checker** * — procesamiento de expresiones
2. **UVa 00397 - Equation Elation *** — realizar la siguiente operación de forma iterativa
3. **UVa 01200 - A DP Problem *** — LA 2972 - Tehran03, trocear la entrada
4. **UVa 10906 - Strange Integration *** — procesamiento BNF, solución iterativa
5. *Kattis - autori ** — problema sencillo de troceo de cadenas
6. *Kattis - pervasiveheartmonitor ** — procesamiento sencillo, después hallar la media
7. *Kattis - timebomb ** — solo es un problema tedioso de procesamiento de la entrada, divisibilidad por 6

Adicionales UVa: *00271*, *00327*, *00391*, *00442*, *00486*, *00537*, *11148*, *12543*, *13047*, *13093*.
Adicionales Kattis: *genealogical*, *tripletexting*.

**Formato de salida, fáciles**

1. Nivel básico: **UVa 00488 - Triangle Wave** * — utilizar varios bucles
2. **UVa 01605 - Building for UN *** — LA 4044 - NortheasternEurope07, podemos responder con solo $h = 2$ niveles
3. **UVa 10500 - Robot maps *** — simular, formato de salida
4. **UVa 12364 - In Braille *** — comprobación de *array* bidimensional, comprobar todos los posibles dígitos [0..9]

5. *Kattis - display ** — `unordered_map`, mostrar dígito → en versión aumentada 7x5
6. *Kattis - musicalnotation ** — sencillo pero tedioso
7. *Kattis - skener ** — aumento de *array* de caracteres bidimensional

Adicionales UVa: *00110*, *00320*, *00445*, *00490*, *10146*, *10894*, *11074*, *11482*, *11965*, *13091*.
Adicionales Kattis: *krizaljka*, *mirror*, *multiplication*, *okvir*, *okviri*.

**Para perder el tiempo, fáciles**

1. Nivel básico: *Kattis - asciiaddition ** — problema de a+b en formato texto, una minucia, para perder el tiempo
2. **UVa 11638 - Temperature Monitoring *** — simulación, necesita usar máscara de bits para el parámetro $C$
3. **UVa 12085 - Mobile Casanova *** — LA 2189 - Dhaka06, cuidado con el veredicto PE
4. **UVa 12608 - Garbage Collection *** — simulación con varios casos límite
5. *Kattis - glitchbot ** — pérdida de tiempo, simulación en $O(n^2)$, no mostrar más de una posible respuesta
6. *Kattis - pachydermpeanutpacking ** — pérdida de tiempo, simulación sencilla de un bucle
7. *Kattis - printingcosts ** — pérdida de tiempo evidente, lo más difícil es procesar los costes de cada carácter en el enunciado del problema

Adicionales UVa: *00144*, *00214*, *00335*, *00349*, *00556*, *10028*, *10033*, *10134*, *10850*, *12060*, *12700*.
Adicionales Kattis: *averagespeed*, *gerrymandering*.

**Para perder el tiempo, difíciles**

1. Nivel básico: **UVa 10188 - Automated Judge Script** * — simulación
2. **UVa 00405 - Message Routing *** — simulación
3. **UVa 11717 - Energy Saving Micro... *** — simulación complicada
4. **UVa 12280 - A Digital Satire of ... *** — un problema tedioso
5. *Kattis - froggie ** — solo una simulación, pero con muchos casos límite, S puede ser 0
6. *Kattis - functionalfun ** — seguir el enunciado, 5 casos, problema de procesamiento tedioso, requiere algún tipo de mapeo
7. *Kattis - windows ** — LA 7162 - WorldFinals Marrakech15, problema de simulación tedioso, también disponible en UVa 01721 - Window Manager

Adicionales UVa: *00337*, *00381*, *00603*, *00618*, *00830*, *00945*, *10142*, *10267*, *10961*, *11140*.
Adicionales Kattis: *interpreter*, *lumbercraft*, *sabor*, *touchdown*.

## 1.7 Soluciones a los ejercicios no resaltados

**Ejercicio 1.1.1**: un caso de prueba sencillo para hacer fallar a algoritmos voraces es $N = 2$, $\{(2,0), (2,1), (0,0), (4,0)\}$. Un algoritmo voraz emparejará de forma incorrecta $\{(2,0), (2,1)\}$ y $\{(0,0), (4,0)\}$ con un coste de 5,00, mientras que la solución óptima es emparejar $\{(0,0), (2,0)\}$ y $\{(2,1), (4,0)\}$ con un coste de 4,24.

**Ejercicio 1.1.2**: para realizar una búsqueda completa ingenua, como la descrita en el texto, son necesarias hasta ${}_{16}C_2 \times_{14} C_2 \times \cdots \times_2 C_2 = \frac{16!}{2!^8} \approx 8 \times 10^{10}$ operaciones para el caso más grande, con $N = 8$, lo que es demasiado. Sin embargo, hay formas de podar el espacio de búsqueda para que la búsqueda completa pueda ser operativa. También puedes intentar el **ejercicio 1.1.3***.

**Ejercicio 1.3.2.1**: la tabla 1.3 completa (sin el UVa 10360) es la siguiente.

| UVa/Kattis | Título | Tipo de problema | Más información |
|---|---|---|---|
| wordcloud | Word Cloud | *Ad hoc* | Sección 1.6 |
| turbo | Turbo | Árbol de Fenwick, RSQ | Sección 2.4 |
| hindex | H-Index | BSTA + búsqueda binaria | Sección 3.3 |
| 11292 | Dragon of Loowater | Voraz (clásico) | Sección 3.4 |
| 11450 | Wedding Shopping | DP (no clásica) | Sección 3.5 |
| 11512 | GATTACA | Cadenas (*array* de sufijos, LRS) | Volumen II |
| 10065 | Useless Tile Packers | Geometría (CH + área del polígono) | Volumen II |
| 11506 | Angry Programmer | Grafos (corte mínimo) | Volumen II |
| bilateral | Bilateral Projects | MVC, bipartito, MCBM | Volumen II |
| carpool | Carpool | APSP, búsqueda completa, DP | Volumen II |

**Ejercicio 1.3.3.1**: las respuestas son:

1. (b) Utilizar un árbol de búsqueda binaria equilibrado como cola de prioridad (sección 2.3).
2. Si la lista `L` es estática, (a) *array* sencillo procesado previamente con programación dinámica (secciones 2.2 y 3.5). Si la lista `L` es dinámica, entonces (d) árbol de Fenwick, es una respuesta mejor (más fácil de implementar que (c) árbol de segmentos).
3. (b) Usar consulta de suma de rango bidimensional (sección 3.5.2).
4. (b) Ver la solución en la sección 3.2.3.
5. (a) Programación dinámica en $O(V + E)$ (secciones 3.5, 4.2.6 y 4.6.1). Sin embargo, también es posible (c) algoritmo de Dijkstra en $O((V + E) \log V)$, ya que el factor adicional $O(\log V)$ sigue siendo 'pequeño' para un valor de $V$ hasta $100K$ y es difícil separar este factor log.
6. (a) Criba de Eratóstenes (Volumen II).
7. (b) La técnica ingenua del punto anterior no funcionará. Ver la fórmula de Legendre en el Volumen II.
8. (b) La técnica ingenua es muy lenta. Usar KMP/*array* de sufijos/Rabin–Karp (Volumen II).
9. (a) Sí, es posible utilizar una búsqueda completa (sección 3.2).

10. (b) No, hay que encontrar otra forma. En primer lugar, buscar la envolvente convexa de los $N$ puntos en $O(n \log n)$ (Volumen II). Digamos que hay $CH(S) = k$ puntos. Como están distribuidos aleatoriamente, $k$ será mucho más pequeño que $N$. Después, encontrar los dos puntos más alejados, examinando todos los pares de puntos de $CH(S)$ en $O(k^2)$.
11. (c) Cuando los puntos pueden no estar distribuidos aleatoriamente, $k$ puede ser $N$, es decir, todos los puntos quedan dentro de la envolvente convexa. Para resolver esta variante, necesitamos utilizar la técnica del calibre giratorio en $O(n)$.

**Ejercicio 1.3.4.1**: mostramos algunas soluciones seleccionas y se pueden encontrar alternativas en `https://github.com/stevenhalim/cpbook-code/tree/master/ch1/Ex_1.3.4.1`:

```
import java.util.*;                          // código Java para la tarea 1
class Main {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    double d = sc.nextDouble();
    System.out.printf("%7.3f\n", d);        // Java también tiene printf
  }
}
```

```
#include <bits/stdc++.h>                     // código C++ para la tarea 2
using namespace std;
int main() {
  int n; scanf("%d", &n);
  printf("%.*lf\n", n, M_PI);                  // ajustar ancho del campo
}
```

```
from datetime import date                   # código Python para la tarea 3
s = date(2010, 8, 9)                          # fecha de publicación de CP1
t = date.today()
print(s.strftime("%a"))                          # 'Mon', %A para 'Monday'
print("{} day(s) ago".format((t-s).days))     # la respuesta va creciendo
```

```
print(*sorted(set(input().split())), sep='\n', key=int) # Python tarea 4
```

```
#include <bits/stdc++.h>                     // código C++ para la tarea 5
using namespace std;
typedef tuple<int, int, int> iii;                  // usar orden natural
int main() {
  vector<iii> birthdays;
  birthdays.emplace_back(5, 24, -1980);          // reordenar DD/MM/AAAA
  birthdays.emplace_back(5, 24, -1982);          // a MM, DD, y entonces
  birthdays.emplace_back(11, 13, -1983);           // usar AAAA NEGATIVO
  sort(birthdays.begin(), birthdays.end());              // y eso es todo
```

```
  for (auto &[mm, dd, yyyy] : birthdays)                // estilo C++17
    printf("%d %d %d\n", dd, mm, -yyyy);
}
```

```
#include <bits/stdc++.h>                    // código C++ para la tarea 6
using namespace std;
int main() {
  int n = 5, L[] = {5, 7, 8, 10, 20}, v = 7;
  sort(L, L+n);
  printf("%d\n", binary_search(L, L+n, v));         // debe ser índice 1
}
```

```
#include <bits/stdc++.h>                    // código C++ para la tarea 7
using namespace std;
int main() {
  int p[10], N = 10;
  for (int i = 0; i < N; ++i) p[i] = i;
  do {
    for (int i = 0; i < N; ++i) printf("%c ", 'A'+p[i]);
    printf("\n");
  }
  while (next_permutation(p, p+N));
}
```

```
#include <bits/stdc++.h>                    // código C++ para la tarea 8
using namespace std;
#define LSOne(S) ((S) & -(S))               // cuidado con los paréntesis
int main() {
  int N = 20;
  for (int i = 0; i < (1<<N); ++i) {
    int pos = i;
    while (pos) {
      int ls = LSOne(pos);
      pos -= ls;
      printf("%d ", __builtin_ctz(ls)); // el índice está en el conjunto
    }
    printf("\n");
  }
}
```

```
import java.math.*;                         // código Java para la tarea 9
class Main {
  public static void main(String[] args) {
    String str = "FF"; int X = 16, Y = 10;
```

```
    System.out.println(new BigInteger(str, X).toString(Y));
  }
}
```

```
class Main {                              // código Java para la tarea 10
  public static void main(String[] args) {
    String S = "line: a70 and z72 will be replaced, aa24 and a872 won't";
    System.out.println(S.replaceAll("\\b+[a-z][0-9][0-9]\\b+", "***"));
  }
}
```

```
import java.math.*;                       // código Java para la tarea 11
class Main {
  public static void main(String[] args) throws Exception {
    BigInteger x = new BigInteger("48112959837082048697"); // BigInteger
    System.out.println(x.isProbablePrime(10) ? "Prime" : "Composite");
  }
}
```

```
print(eval(input()))                     # código Python para la tarea 12
```

**Ejercicio 1.3.5.1**: las consideraciones situacionales están entre paréntesis:

1. Obtienes un veredicto de WA en un problema muy sencillo. ¿Qué deberías hacer?
   *a*) Abandonar este problema y seguir con otro. **(No sirve, tu equipo perderá.)**
   *b*) Mejorar el rendimiento de tu solución (optimizando el código/algoritmo). **(No es útil.)**
   *c*) Volver al leer el enunciado cuidadosamente. **(Buena idea.)**
   *d*) Crear casos de prueba complejos para tratar de encontrar el error. **(La respuesta más lógica.)**
   *e*) Si la competición es por equipos, pedir a un compañero que trate de resolverlo. **(Esta opción sería factible, ya que podría haber asumido elementos del problema de forma equivocada. Por lo tanto, no deberías comentar los detalles con el compañero que lo vaya a resolver. Aun así, tu equipo perderá un tiempo precioso.)**
2. Recibes un veredicto TLE para tu solución $O(N^3)$. Sin embargo, el máximo de $N$ es 100. ¿Qué deberías hacer?
   *a*) Abandonar este problema y seguir con otro. **(No sirve, tu equipo perderá.)**
   *b*) Mejorar el rendimiento de tu solución (mejor código/algoritmo). **(No sirve, no deberíamos obtener un veredicto TLE con un algoritmo $O(N^3)$ si $N \leq 400$.)**
   *c*) Crear casos de prueba complejos para tratar de encontrar el error. **(Esta es la respuesta, pues es posible que el programa esté entrando en algún bucle infinito con algún caso de prueba.)**

3. Ampliación a la pregunta anterior: ¿qué ocurre si el máximo de $N$ es 100 000?
   **(Si $N > 400$, puede que no tengas más opción que mejorar el rendimiento del algoritmo actual, o utilizar otro más rápido. Para empezar, no deberías haber enviado el código.)**

4. Otra ampliación a la pregunta anterior: ¿qué ocurre si el máximo de $N$ es 5000, la salida solo depende del tamaño de la entrada $N$ y todavía quedan *cuatro horas* de concurso?
   **(Si la salida solo depende de $N$, es *posible* que logres calcular previamente todas las soluciones posibles ejecutando tu algoritmo en $O(N^3)$ durante algunos minutos, mientras tus compañeros utilizan el ordenador. Una vez que la solución en $O(N^3)$ haya terminado, tendrás todas las respuestas. Podrás enviar la respuesta en $O(1)$ siempre que no superes el tamaño máximo de código establecido por el juez.)**

5. Recibes un veredicto de RTE. El código, aparentemente, se ejecuta perfectamente en tu sistema local. ¿Qué deberías hacer?
   **(La causas más comunes de los RTE suelen ser el tamaño demasiado pequeño de los *arrays* o errores de desbordamiento de pila o recursión infinita. Diseña casos de prueba que puedan provocar esos errores en tu código.)**

6. Treinta minutos después de comenzar el concurso, le echas un vistazo al marcador. Hay *muchos* equipos que han resuelto el problema *X*, que tu equipo todavía no ha intentado. ¿Qué deberías hacer?
   **(Uno de los miembros del equipo debería trabajar inmediatamente en el problema *X*, ya que podría ser relativamente sencillo. Una situación así no es una buena noticia, ya que limitará tus posibilidades de obtener una buena clasificación.)**

7. A mitad del concurso le echas un vistazo al marcador. El equipo líder (asumiendo que no sea el tuyo) acaba de resolver el problema *Y*. ¿Qué deberías hacer?
   **(Si tu equipo no es quien está 'marcando el ritmo', es una buena idea 'ignorar' lo que está haciendo el equipo líder y concentrarse en aquellos problemas que tengas identificados como 'resolubles'. A esas alturas del concurso, tu equipo ya debería haber leído todos los problemas e identificado aquellos que resultan más asequibles para vuestras capacidades.)**

8. Tu equipo le ha dedicado dos horas a un problema complicado. Ya habéis enviado varias implementaciones realizadas por diferentes miembros del equipo. Todos los envíos han resultado incorrectos. No tenéis ni idea de dónde está el fallo. ¿Qué deberías hacer?
   **(Es el momento de abandonar este problema. No acapares el ordenador y deja que un compañero resuelva otro problema. O tu equipo no ha entendido el problema o, en casos muy excepcionales, la solución del juez es incorrecta. Sea como sea, la situación no pinta nada bien.)**

9. Queda una hora para que termine el concurso. Tienes un código con veredicto WA y una idea nueva para *otro* problema. ¿Qué deberías hacer?
   **(Utilizando terminología de ajedrez, estás en una situación de 'final'.)**

   *a*) Abandonar el problema con el código WA e intentar el otro problema para tratar de resolver uno más. **(Opción válida en concursos individuales, como la IOI.)**

   *b*) Insistir en depurar el código WA. No queda tiempo para comenzar a trabajar en un nuevo problema. **(Si la idea para otro problema implica una programación compleja y tediosa, puede ser buena idea centrarse en el código WA en vez de terminar con dos soluciones incompletas o no aceptadas.)**

*c*) En el ICPC, imprimir el código WA y pedir a los otros dos miembros del equipo que lo analicen, mientras tú comienzas a trabajar en el problema nuevo, en un intento de resolver *dos* problemas más. **(Si la solución del otro problema se puede programar en menos de 30 minutos, puedes implementarla mientras tus compañeros tratan de encontrar el error del código WA, trabajando sobre copias impresas.)**

**Ejercicio 1.3.5.2**:

1. `#define LSOne(S) (S & -S)` provocará un error muy difícil de detectar, como: `(7-5 & -7-5) = (2 & -12) = 0`.
   Utiliza `#define LSOne(S) ((S) & -(S))` en su lugar y calcula: `(7-5) & -(7-5) = 2 & -2 = 2`.
2. `__builtin_ctz(v)` es para enteros de 32 bits, utiliza `__builtin_ctzll(v)` para enteros de 64 bits.
3. Hacerlo eliminará todas las copias de *v*. En su lugar, utiliza `ms.erase(ms.find(v))`.
4. El iterado es invalidado cuando el `vector` debe doblar su tamaño para reubicar su contenido. Cuidado con esos casos sutiles de invalidación de iteradores.
5. Igualmente, ten cuidado cuando utilices el símbolo de referencia `&`, pues también puede provocar errores.

## Soluciones en C para la sección 1.5

**Ejercicio 1.5.1**:

a) Una cadena se almacena como un *array* de caracteres terminado con el carácter `NULL`, por ejemplo, `char str[30*10+50], line[30+50];`. Es una buena práctica declarar un tamaño de *array* un poco superior al necesario, para evitar el error de "pasarse por uno".

b) Para leer la entrada línea a línea, utilizamos[43] `gets(line);` o `fgets(line, 40, stdin);`, incluidos en la biblioteca `string.h` (o `cstring`).

c) Establecemos, inicialmente, `str` como una cadena vacía y, a continuación, combinamos las líneas leídas en una cadena más grande, utilizando la función `strcat`. Si la línea actual no es la última, añadimos un espacio en blanco al final de `str`, para que la última palabra no quede unida a la primera de la siguiente línea.

d) Detenemos la lectura de la entrada cuando `strncmp(line, ".......", 7) == 0`. Hay que tener en cuenta que `strncmp(str1, str2, num)` solo compara los primeros `num` caracteres.

**Ejercicio 1.5.2**:

a) Para encontrar una subcadena dentro de una cadena relativamente corta (la versión estándar del problema de coincidencia de cadenas), nos basta la función de la biblioteca. Podemos utilizar `p = strstr(str, substr);`. El valor de `p` será `NULL` si `substr` no aparece dentro de `str`.

[43] La función `gets` no es segura, porque no realiza comprobaciones de límites en el tamaño de la entrada.

b) Si hay varias copias de `substr` dentro de `str`, podemos utilizar `pos = strstr(str+pos, substr)`. Empezamos con `pos = 0`, es decir, buscamos desde el primer carácter de `str`. Una vez hayamos encontrado una aparición de `substr` en `str`, podemos llamar a `pos = strstr(str+pos, substr)` nuevamente, pero esta vez `pos` será el índice de la aparición actual de `substr` en `str` *más* 1, para obtener la siguiente aparición. Repetiremos el proceso hasta que `pos == NULL`. Esta solución en C requiere entender el direccionamiento de memoria de un *array* en C.

**Ejercicio 1.5.3**: en muchas tareas de procesamiento de cadenas, nos vemos en la necesidad de iterar una vez sobre todos los caracteres de `str`. Si hay *n* caracteres en `str`, esta búsqueda supone $O(n)$. Tanto en C como en C++, podemos utilizar `tolower(ch)` y `toupper(ch)`, de `ctype.h`, para convertir un carácter a su versión minúscula o mayúscula. También disponemos de `isalpha(ch)/isdigit(ch)`, para verificar si un carácter determinado es alfabético (`[A-Za-z]`) o numérico, respectivamente. Para comprobar si un carácter es una vocal, un método puede ser crear una cadena `vocales = "aeiou";` y verificar si está contenido en ella. Si queremos saber si es consonante, basta comprobar que el carácter es alfabético pero no una vocal.

**Ejercicio 1.5.4**: soluciones combinadas en C y C++:

a) Una forma muy sencilla de trocear una cadena es con `strtok(str, delimiters);` en C o `stringstreams` en C++.

b) Los segmentos se pueden almacenar en un `vector<string> tokens` de C++.

c) Podemos utilizar `sort(primero, último)` de la STL de C++ para ordenar `vector<string> tokens`. Si es necesario, podemos convertir una `string` de C++ a C utilizando `str.c_str()`.

**Ejercicio 1.5.5**: ver la solución en C++.

**Ejercicio 1.5.6**: leer la entrada carácter por carácter y contarlos, buscar la presencia de '`\n`', que indica el final de línea. Reservar previamente un espacio de almacenamiento de tamaño fijo no es una buena idea, ya que el autor del problema podría utilizar cadenas extraordinariamente largas con el objetivo de hacer fallar el código de las soluciones.

## Soluciones en C++ para la sección 1.5

**Ejercicio 1.5.1**:

a) Podemos utilizar la clase `string`.

b) Podemos utilizar `getline(cin, string_name);`.

c) Podemos utilizar directamente el operador '+' para concatenar cadenas.

d) Podemos utilizar `string_name.rfind(".......", 0) == 0`.

**Ejercicio 1.5.2**:

a) Podemos utilizar la función `find(str)` de la clase `string`.

b) La misma idea que en C. Podemos establecer el valor de desplazamiento en el segundo parámetro de la función `find(str, pos)` de la clase `string`.

**Ejercicio 1.5.3-4**: las mismas soluciones que en C.

**Ejercicio 1.5.5**: podemos utilizar `unordered_map<string, int>` de la STL de C++ para mantener un registro de la frecuencia de cada palabra. Cada vez que encontremos un nuevo trozo (que es una cadena), incrementamos en uno la frecuencia correspondiente de ese trozo. Por último, examinamos todos los trozos y determinamos el que tiene mayor frecuencia. Lo trataremos en la sección 2.3.

**Ejercicio 1.5.6**: la misma solución que en C, o utilizar la clase de longitud flexible `string`.

## Soluciones en Python para la sección 1.5

**Ejercicio 1.5.1**:

a) Almacenar la cadena en una variable de Python.

b) Podemos utilizar el método `input()` para leer una línea.

c) Podemos utilizar directamente el operador '+' para concatenar cadenas.

d) Podemos utilizar el método `startswith(prefijo)`.

**Ejercicio 1.5.2**:

a) Podemos utilizar la función `find(sub)` de una cadena.

b) La misma idea en que C. Podemos indicar el valor de desplazamiento en el segundo parámetro de la función `find(sub, inicio)` de una cadena.

**Ejercicio 1.5.3**: podemos utilizar `lower()` para convertir una cadena a su versión en minúsculas. En muchas tareas de procesamiento de cadenas, nos vemos obligados a iterar por cada carácter de `str` una vez. Si `str` tiene $n$ caracteres, esa exploración requiere $O(n)$. Podemos utilizar la operación ternaria de Python `1 if (c in el_digito) else 0` donde `el_digito = list("0123456789")`. La misma técnica sirve para contar el número de letras y de vocales.

**Ejercicio 1.5.4**:

a) Podemos utilizar el método `split(separador)`, por ejemplo,

```
token = "jovencillo emponzoñado de whisky".split(" ");
```

b) Podemos utilizar `list` (ver el caso anterior).

c) Podemos utilizar `tokens.sort()`.

**Ejercicio 1.5.5**: la misma solución que en C++, pero utilizamos `freq = defaultdict(int)` después de la llamada `from collections import defaultdict`. Lo veremos en la sección 2.3.

**Ejercicio 1.5.6**: no es fácil leer la entrada carácter por carácter en Python, por lo que la cargaremos entera en memoria mediante `longline = input()` (Python ajusta automáticamente el tamaño del *buffer*) e informaremos de la longitud.

## Soluciones en Java para la sección 1.5

**Ejercicio 1.5.1**:

a) Podemos utilizar las clases `String`, `StringBuffer` o `StringBuilder` (esta es más rápida que `StringBuffer`).

b) Podemos utilizar el método `nextLine()` de `Scanner` de Java. Para conseguir una E/S más rápida, podemos considerar utilizar el método `readLine()` de `BufferedReader` de Java.

c) Podemos utilizar el método `append(str)` de `StringBuilder`. No debemos concatenar cadenas en Java con el operador '+', ya que la clase `String` de Java es inmutable y, por ello, esa operación es (muy) costosa.

d) Podemos utilizar el método `startsWith(str)` de `String` de Java.

**Ejercicio 1.5.2**:

a) Podemos utilizar el método `indexOf(str)` de la clase `String`.

b) La misma idea que en C. Podemos establecer el valor de desplazamiento en el segundo parámetro del método `indexOf(str, desdeÍndice)` de la clase `String`.

**Ejercicio 1.5.3**: utilizar las clases `StringBuilder` y `Character` de Java para estas operaciones.

**Ejercicio 1.5.4**:

a) Podemos utilizar la clase `StringTokenizer` o el método `split(regex)` de la clase `String` de Java.

b) Podemos utilizar un `ArrayList` de `String`.

c) Podemos utilizar `Collections.sort`.

**Ejercicio 1.5.5**: la misma idea que en C++. Podemos utilizar `HashMap<String, Integer>`. Lo trataremos en la sección 2.3.

**Ejercicio 1.5.6**: necesitamos utilizar el método `read()` de la clase `BufferedReader` de Java.

## Soluciones en OCaml para la sección 1.5

**Ejercicio 1.5.1**:

a) Podemos utilizar `string`.

b) Podemos utilizar `read_line()` del módulo `Stdlib`.

c) Podemos utilizar `concat` del módulo `String`.

d) Podemos utilizar la comprobación mediante expresiones regulares.

**Ejercicio 1.5.2**:

a) Podemos utilizar `search_forward` del módulo `Str`.

b) Podemos ajustar el parámetros `start` de `search_forward`.

**Ejercicio 1.5.3**: podemos utilizar `lowercase_ascii` para convertir la cadena de entrada a minúsculas. Después, podemos utilizar el iterador `to_seq` para iterar por la cadena y emplear una función anónima que, al encontrar un `['0'..'9']` incremente el número de dígitos y, si encuentra una `['a'..'z']`, lo sume a las vocales ("`aeiou`") o consonantes, según corresponda.

**Ejercicio 1.5.4**:

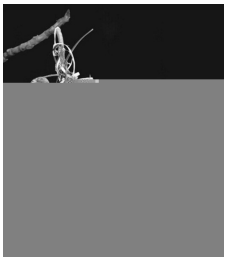
a) Podemos utilizar `split` del módulo `Str`, especificando la expresión regular requerida para la partición.

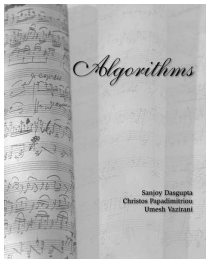
b) Podemos utilizar el módulo `List`.

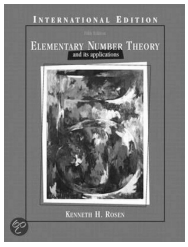
c) Podemos utilizar `sort` del módulo `List`.

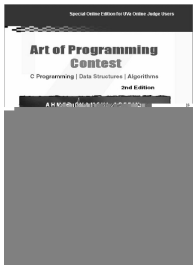
**Ejercicio 1.5.5**: podemos utilizar `Hashtbl`. Lo trataremos en la sección 2.3.

**Ejercicio 1.5.6**: lo cargaremos todo en memoria mediante `let longline = read_line() in` (OCaml ajusta el tamaño de *buffer* automáticamente) e informaremos de la longitud.






'JHVSB"MHVOBT EF MBT GVFOUFT RVF IBO JOTQJSBEP B MPT

4PMVDJPOFT B MPT FKFSDJD